SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

CYB23CL302 - MACHINE LEARNING CLOUD SERVICES LABORATORY

LABORATORY REPORT

Submitted by

G L SWAMINATHAN – E0224033

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Cybersecurity and Internet of Things)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -600116

October, 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

CYB23CL302 - MACHINE LEARNING CLOUD SERVICES LABORATORY

LABORATORY REPORT

Submitted by

G L SWAMINATHAN – E0224033

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Cybersecurity and Internet of Things)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -600116

October, 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this laboratory report **CYB23CL302 - Machine Learning Cloud Services Laboratory** is the bonafide record of work done by **G L SWAMINATHAN – E0224033** who carried out the laboratory work under my supervision.

Course Faculty

Dr. Jayanthi G
Associate Professor,
Department of Cyber Security and Internet of Things
Sri Ramachandra Faculty of Engineering and Technology,
SRIHER, Porur, Chennai-600 116.

Evaluation Date: 27-10-2026

Signature of Internal Examiner **Signature of External Examiner**

TABLE OF CONTENTS

Exp. No	Date	Title of the Experiment	Page. No
1	14-07-2026	Review of NumPy and Pandas	1
2	21-07-2026	NumPy: Data Preparation for Machine Learning using Malware Analysis Dataset	6
3.a	28-07-2026	Supervised Learning Regression: Prediction of Detection time of Cyber incidents	13
3.b	04-08-2026	Investment on Advertising Media versus Sales	21
4.a	11-08-2026	Decision Tree Classification	27
4.b.1	18-08-2026	Supervised Learning Classification : Random Forest using Wine Quality dataset	33
4.b.2	18-08-2026	Supervised Learning Classification : Random Forest using Digit dataset	36
5.a	25-08-2026	Estimate the Optimal Number of Clusters	39
5.b	25-08-2026	Text Data Clustering using K means	41
5.c	01-09-2026	Association Rule Mining	48
6	08-09-2026	The study of Reinforcement Learning Techniques	51
7.a	15-09-2026	TF-IDF and Cosine Similarity for comparing two documents	58
7.b	15-09-2026	Case study on TF-IDF and Cosine Similarity : Tweets dataset (Kaggle)	65
8	22-09-2026	Text Classification using Multinomial Naïve Bayes Algorithm	68
9	29-09-2026	Machine Learning on AWS	72
10.a	06-10-2026	Serialization of ML models	78
10.b	06-10-2026	Continuous Deployment and Continuous Integration of ML Models	84
11	13-10-2026	Project Work – Data Dictionary and Implementation with results and discussions	

Exp. No: 1	Review of NumPy and Pandas
Date: 14-07-2026	

AIM:

To review the fundamental features of the NumPy and Pandas libraries of Python, which are used for numerical computing and data handling in machine learning.

OBJECTIVE

1. To create NumPy arrays of one or more dimensions and inspect their attributes.
2. To access elements by indexing and to reshape arrays using np.arange() and reshape().
3. To compare Python lists with NumPy arrays and perform vectorised arithmetic.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3)
- Jupyter Notebook
- NumPy library

DESCRIPTION

NumPy (Numerical Python) is the core library for scientific computing in Python. Its central object is the **ndarray**, a fixed-type, N-dimensional array that stores data in contiguous memory and supports fast, vectorised operations. Every array has a number of attributes: ndim (number of dimensions), shape (size along each dimension), size (total number of elements) and dtype (data type of the elements). Functions such as np.arange(start, stop, step) generate evenly spaced values, and reshape() changes the dimensions of an array without changing its data.

Pandas is built on NumPy and provides the labelled **Series** and **DataFrame** structures, with tools for reading files, inspecting data (head(), info(), describe()) and handling missing values. NumPy arrays and DataFrames are the standard inputs to scikit-learn.

PROCEDURE

1. Launch Jupyter Notebook from Anaconda and import NumPy as np.
2. Create 1-D, 2-D and 3-D arrays and display their size, shape, ndim and dtype.
3. Access the first and last elements of arrays using indices 0 and -1.
4. Compare np.size() on a Python list with the .size attribute of an array, and add a scalar to an array.

5. Complete NumPy Worksheet-1 by generating sequences with np.arange() and reshaping them.

CODE

Activity-1_Overview_of_NumPy.ipynb

In [1]:
```
import numpy as np
A = np.array([1,2,3])
```

In [3]:
```
import numpy as np
A=np.array([1,22,-3,14,50])
print(A)
```

In [4]:
```
B = np.array([[10,22,30],[11,14,7]])
B
```

In [5]:
```
print (A.size)
print (np.size(A))
```

In [7]:
```
print(B.size)
print(np.size(B))
```

In [8]:
```
array_nd = np.arange(24).reshape(2,3,4)
print(array_nd)
```

In [11]:
```
print("Shape: ",array_nd.shape)
print("Dimension: ",array_nd.ndim)
```

In [12]:
```
names = np.array(["Alice","Bob","David","Eva"])
print(names)
print(names.size)
print(np.size(names))
```

In [13]:
```
first_element = names[0]
last_element = names[-1]
first_element
```

In [14]:
```
last_element
```

In [16]:
```
arr_2d = np.array([["apple","guava"],["Mango","Orange"]])
first_elem = arr_2d[0,0]
last_elem = arr_2d[-1,-1]
first_elem
```

In [17]:
```
last_elem
```

In [18]:
```
import numpy as np
my_list = [1,2,3]
my_arr = np.array([1,2,3])
```

```
print(np.size(my_list))
print(np.size(my_arr))
print(my_arr.size)

print(my_list.size)
```

In [20]:
```
matrix = np.array([[1,2,3],[4,5,6]])
print(matrix.size)
print(np.size(matrix,axis=0))
print(np.size(matrix,axis=1))
```

In [22]:
```
import numpy as np
B = np.array([[1,2],[3,4]])
print(B.ndim)
print(B.shape)
print(B.size)
print(B.dtype)
```

In [23]:
```
A = np.array([1,2,3])
print(A+5)
```

NumPy Worksheet-1.ipynb

In [19]:
```
import numpy as np
arr = np.array([0,1,2,3,4,5,6,7,8,9])
print(arr)
A = np.arange(10)
print(A)
```

In [20]:
```
import numpy as np
arr = np.array([5,10,15,20,25])
print(arr)
A = np.arange(5,26,5)
print(A)
```

In [21]:
```
import numpy as np
arr = np.array([20,18,16,14,12])
print(arr)
A = np.arange(20,10,-2)
print(A)
```

In [22]:
```
import numpy as np
A = np.arange(12).reshape(3,4)
print(A)
```

In [23]:
```
import numpy as np
A = np.arange(10,35).reshape(5,5)
print(A)
```

In [24]:
```
import numpy as np
arr = np.arange(24).reshape(2,3,4)
print(arr)
```

In [25]:

```
import numpy as np
arr = np.arange(100,172,2).reshape(2,2,3,3)
print(arr)
```

In [26]:

```
import numpy as np
mystery_arr = np.arange(1, 10, 2).reshape(1, 5)
print(mystery_arr)
```

OUTPUT

Activity-1_Overview_of_NumPy.ipynb

```
[3]:  [ 1 22 -3 14 50]

[4]:  array([[10, 22, 30],
             [11, 14,  7]])

[5]:  5
      5

[7]:  6
      6

[8]:  [[[ 0  1  2  3]
        [ 4  5  6  7]
        [ 8  9 10 11]]

       [[12 13 14 15]
        [16 17 18 19]
        [20 21 22 23]]]

[11]: Shape:  (2, 3, 4)
      Dimension:  3

[12]: ['Alice' 'Bob' 'David' 'Eva']
      4
      4

[13]: np.str_('Alice')

[14]: np.str_('Eva')

[16]: np.str_('apple')

[17]: np.str_('Orange')
```

JUPYTER NOTEBOOK OUTPUT – IN [3], [4], [5], [7], [8], [11], [12], [13], [14], [16], [17]

```
[18]: 3
      3
      3
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[18], line 9
      6 print(np.size(my_arr))
      7 print(my_arr.size)
----> 9 print(my_list.size)

AttributeError: 'list' object has no attribute 'size'

[20]: 6
      2
      3

[22]: 2
      (2, 2)
      4
      int64

[23]: [6 7 8]
```

JUPYTER NOTEBOOK OUTPUT – IN [18], [20], [22], [23]

NumPy Worksheet-1.ipynb

```
[19]: [0 1 2 3 4 5 6 7 8 9]
      [0 1 2 3 4 5 6 7 8 9]

[20]: [ 5 10 15 20 25]
      [ 5 10 15 20 25]

[21]: [20 18 16 14 12]
      [20 18 16 14 12]

[22]: [[ 0  1  2  3]
       [ 4  5  6  7]
       [ 8  9 10 11]]

[23]: [[10 11 12 13 14]
       [15 16 17 18 19]
       [20 21 22 23 24]
       [25 26 27 28 29]
       [30 31 32 33 34]]

[24]: [[[ 0  1  2  3]
        [ 4  5  6  7]
        [ 8  9 10 11]]

       [[12 13 14 15]
        [16 17 18 19]
        [20 21 22 23]]]
```

JUPYTER NOTEBOOK OUTPUT – IN [19], [20], [21], [22], [23], [24]

```
[25]: [[[[100 102 104]
         [106 108 110]
         [112 114 116]]

        [[118 120 122]
         [124 126 128]
         [130 132 134]]]


       [[[136 138 140]
         [142 144 146]
         [148 150 152]]

        [[154 156 158]
         [160 162 164]
         [166 168 170]]]]

[26]: [[1 3 5 7 9]]
```

JUPYTER NOTEBOOK OUTPUT – IN [25], [26]

RESULT

Arrays of one to four dimensions were created and inspected; for example, np.arange(24).reshape(2,3,4) has shape (2, 3, 4) and 3 dimensions, and adding 5 to [1 2 3] gave [6 7 8]. Calling .size on a Python list raised an AttributeError, confirming that this attribute exists only for NumPy arrays. All worksheet sequences were generated correctly.

CONCLUSION

The experiment reviewed NumPy array creation, attributes, indexing, reshaping and vectorised arithmetic, which form the foundation for data preparation in the later experiments.

Exp. No: 2	NumPy: Data Preparation for Machine Learning using Malware Analysis Dataset
Date: 21-07-2026	

AIM:

To prepare a labelled malware analysis dataset for machine learning using NumPy, by separating features and labels, computing statistics, normalising the features and splitting the data into training and testing sets.

OBJECTIVE

1. To create a malware analysis dataset as a NumPy array and inspect its dimensions.
2. To separate features and labels, check for missing values and compute statistics.
3. To normalise the features, filter samples and split the data into training and testing sets.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3)
- Jupyter Notebook
- NumPy library

DESCRIPTION

Each executable file in the dataset is assumed to have been analysed using **static malware analysis**. Every row is one file and the six columns are:

Column	Feature	Meaning
0	File Size	Size of the executable
1	API Calls	Number of suspicious API calls
2	Entropy	Randomness of the file content
3	Sections	Number of PE (Portable Executable) sections – the main chunks of a Windows program that store code and data
4	Imports	Number of imported DLL functions
5	Label	0 = Benign, 1 = Malware

Min-Max scaling, X_norm = (X − X_min) / (X_max − X_min), brings every feature to the range [0, 1]. Shuffling removes any ordering in the data, and a train/test split keeps unseen samples for evaluation.

PROCEDURE

1. Import NumPy and create the 10 × 6 malware dataset.

2. Display the dimensions, shape, size and data type, and separate X (columns 0–4) and y (column 5).
3. Check for missing values and compute the column-wise mean, minimum, maximum and standard deviation.
4. Normalise the features using Min-Max scaling and apply Boolean filtering.
5. Shuffle the dataset (np.random.seed(40)) and split it into 80% training and 20% testing data.

CODE

Exercise-2_Dataset.ipynb

In [ ]: 1. Import numpy

```
import numpy as np
```

In [1]: Step 2: Malware Analysis Dataset

```
import numpy as np
# A sample synthetic dataset is provided below for data preparation
# Labelled Dataset
# How to create a dataset using numpy array?
malware_data = np.array([
[250,15,5.4,4,120,0],
[980,120,7.8,8,340,1],
[450,42,6.2,5,180,0],
[1500,230,7.9,9,510,1],
[320,28,5.9,4,150,0],
[1750,260,8.2,10,580,1],
[700,80,6.8,6,250,1],
[410,30,5.7,4,160,0],
[1250,180,7.5,8,420,1],
[280,20,5.5,4,135,0]
])
```

In [6]: Step 3: Check Dataset Dimensions

```
print("Dimensions: ",malware_data.ndim)
print("Shape: ",malware_data.shape)
print("Size: ",malware_data.size)
print("Data type: ",malware_data.dtype)
```

In [7]: Step 4: Separate Features (X) and Target (y)

```
# Print all rows and first 5 columns (slicing an array)
x = malware_data[:,0:5]
x
```

In [12]:

```
y = malware_data[:,5] #all rows of the last column(5th)
y
```

In [16]:
```
print(x)
```

In [17]:
```
print(y)
```

In [19]: Step 5: View Individual Features
```
file_size = x[:,0]
```

In [20]:
```
print(file_size)
```

In [23]: Entropy
```
entropy = x[:,2]
entropy
```

In [24]: Step 6: Check for Missing Values
```
print(np.isnan(x))
```

In [25]: Count Missing Values
```
print(np.sum(np.isnan(x)))
```

In [26]: Step 7: Compute Dataset Statistics
```
print("Mean: ", np.mean(x,axis = 0)) # axis = 0 means row
```

In [27]: Minimum
```
print(np.min(x,axis=0))
```

In [28]: Maximum
```
print(np.max(x,axis=0))
```

In [29]: Standard Deviation
```
print(np.std(x,axis=0))
```

In [30]: Step 8: Normalize the Features (Min-Max Scaling)
```
x_min = np.min(x,axis = 0)

x_max = np.max(x,axis = 0)

x_norm = (x-x_min)/(x_max-x_min)

print(x_norm)
```

In [32]: Step 9: Boolean Filtering
```
high_entropy = x[:,2] > 7
print(high_entropy)
```

In [33]: Step 10: Find Malware Samples Only
```
malware_data_only = malware_data[malware_data[:,5] == 1]
malware_data_only
```

In [34]: Step 11: Find Benign Files
```
benign_data_only = malware_data[malware_data[:,5] == 0]
benign_data_only
```

In [35]:
```
np.random.seed(40)
np.random.shuffle(malware_data)
print(malware_data)
```

In [36]: Step 13: Split into Training and Testing Data
```
x = malware_data[:,0:5]
y = malware_data[:,5]
# 80% is training size, 20% testing size
train_size = int(0.8 * len(x))
x_train = x[:train_size]
```

```
x_test = x[train_size:]

y_train = y[:train_size]
y_test = y[train_size:]
```

In [37]: Display shapes

```
# Display shapes
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

In [38]: Step 14: Dataset Ready for Machine Learning

```
# Step 14: Dataset Ready for Machine Learning
print("Training Features")

print(x_train)

print()

print("Training Labels")

print(y_train)

print()

print("Testing Features")

print(x_test)

print()

print("Testing Labels")

print(y_test)
```

OUTPUT

```
[6]: Dimensions:  2
     Shape:  (10, 6)
     Size:  60
     Data type:  float64

[7]: array([[ 250. ,   15. ,    5.4,    4. ,  120. ],
            [ 980. ,  120. ,    7.8,    8. ,  340. ],
            [ 450. ,   42. ,    6.2,    5. ,  180. ],
            [1500. ,  230. ,    7.9,    9. ,  510. ],
            [ 320. ,   28. ,    5.9,    4. ,  150. ],
            [1750. ,  260. ,    8.2,   10. ,  580. ],
            [ 700. ,   80. ,    6.8,    6. ,  250. ],
            [ 410. ,   30. ,    5.7,    4. ,  160. ],
            [1250. ,  180. ,    7.5,    8. ,  420. ],
            [ 280. ,   20. ,    5.5,    4. ,  135. ]])

[12]: array([0., 1., 0., 1., 0., 1., 1., 0., 1., 0.])

[16]: [[ 250.    15.     5.4    4.   120. ]
       [ 980.   120.     7.8    8.   340. ]
       [ 450.    42.     6.2    5.   180. ]
       [1500.   230.     7.9    9.   510. ]
       [ 320.    28.     5.9    4.   150. ]
       [1750.   260.     8.2   10.   580. ]
       [ 700.    80.     6.8    6.   250. ]
       [ 410.    30.     5.7    4.   160. ]
       [1250.   180.     7.5    8.   420. ]
       [ 280.    20.     5.5    4.   135. ]]

[17]: [0. 1. 0. 1. 0. 1. 1. 0. 1. 0.]
```

JUPYTER NOTEBOOK OUTPUT – IN [6], [7], [12], [16], [17]

```
[20]: [ 250.  980.  450. 1500.  320. 1750.  700.  410. 1250.  280.]

[23]: array([5.4, 7.8, 6.2, 7.9, 5.9, 8.2, 6.8, 5.7, 7.5, 5.5])

[24]: [[False False False False False]
       [False False False False False]
       [False False False False False]
       [False False False False False]
       [False False False False False]
       [False False False False False]
       [False False False False False]
       [False False False False False]
       [False False False False False]
       [False False False False False]]

[25]: 0

[26]: Mean:  [789.   100.5    6.69   6.2  284.5 ]

[27]: [250.   15.    5.4   4.  120. ]

[28]: [1750.   260.     8.2   10.   580. ]

[29]: [520.96928892  87.61877653   1.02805642   2.22710575 159.60028195]
```

JUPYTER NOTEBOOK OUTPUT – IN [20], [23], [24], [25], [26], [27], [28], [29]

```
[30]: [[0.         0.         0.         0.         0.        ]
       [0.48666667 0.42857143 0.85714286 0.66666667 0.47826087]
       [0.13333333 0.11020408 0.28571429 0.16666667 0.13043478]
       [0.83333333 0.87755102 0.89285714 0.83333333 0.84782609]
       [0.04666667 0.05306122 0.17857143 0.         0.06521739]
       [1.         1.         1.         1.         1.        ]
       [0.3        0.26530612 0.5        0.33333333 0.2826087 ]
       [0.10666667 0.06122449 0.10714286 0.         0.08695652]
       [0.66666667 0.67346939 0.75       0.66666667 0.65217391]
       [0.02       0.02040816 0.03571429 0.         0.0326087 ]]

[32]: [False  True False  True False  True False False  True False]

[33]: array([[9.80e+02, 1.20e+02, 7.80e+00, 8.00e+00, 3.40e+02, 1.00e+00],
             [1.50e+03, 2.30e+02, 7.90e+00, 9.00e+00, 5.10e+02, 1.00e+00],
             [1.75e+03, 2.60e+02, 8.20e+00, 1.00e+01, 5.80e+02, 1.00e+00],
             [7.00e+02, 8.00e+01, 6.80e+00, 6.00e+00, 2.50e+02, 1.00e+00],
             [1.25e+03, 1.80e+02, 7.50e+00, 8.00e+00, 4.20e+02, 1.00e+00]])

[34]: array([[250. ,  15. ,   5.4,   4. , 120. ,   0. ],
             [450. ,  42. ,   6.2,   5. , 180. ,   0. ],
             [320. ,  28. ,   5.9,   4. , 150. ,   0. ],
             [410. ,  30. ,   5.7,   4. , 160. ,   0. ],
             [280. ,  20. ,   5.5,   4. , 135. ,   0. ]])
```

JUPYTER NOTEBOOK OUTPUT – IN [30], [32], [33], [34]

```
[35]: [[3.20e+02 2.80e+01 5.90e+00 4.00e+00 1.50e+02 0.00e+00]
       [1.50e+03 2.30e+02 7.90e+00 9.00e+00 5.10e+02 1.00e+00]
       [1.25e+03 1.80e+02 7.50e+00 8.00e+00 4.20e+02 1.00e+00]
       [9.80e+02 1.20e+02 7.80e+00 8.00e+00 3.40e+02 1.00e+00]
       [4.50e+02 4.20e+01 6.20e+00 5.00e+00 1.80e+02 0.00e+00]
       [2.80e+02 2.00e+01 5.50e+00 4.00e+00 1.35e+02 0.00e+00]
       [2.50e+02 1.50e+01 5.40e+00 4.00e+00 1.20e+02 0.00e+00]
       [1.75e+03 2.60e+02 8.20e+00 1.00e+01 5.80e+02 1.00e+00]
       [4.10e+02 3.00e+01 5.70e+00 4.00e+00 1.60e+02 0.00e+00]
       [7.00e+02 8.00e+01 6.80e+00 6.00e+00 2.50e+02 1.00e+00]]

[37]: (8, 5)
      (2, 5)
      (8,)
      (2,)
```

JUPYTER NOTEBOOK OUTPUT – IN [35], [37]

```
[38]: Training Features
      [[ 320.    28.     5.9    4.   150. ]
       [1500.   230.     7.9    9.   510. ]
       [1250.   180.     7.5    8.   420. ]
       [ 980.   120.     7.8    8.   340. ]
       [ 450.    42.     6.2    5.   180. ]
       [ 280.    20.     5.5    4.   135. ]
       [ 250.    15.     5.4    4.   120. ]
       [1750.   260.     8.2   10.   580. ]]

      Training Labels
      [0. 1. 1. 1. 0. 0. 0. 1.]

      Testing Features
      [[410.   30.    5.7   4.  160. ]
       [700.   80.    6.8   6.  250. ]]

      Testing Labels
      [0. 1.]
```

STEP 14: DATASET READY FOR MACHINE LEARNING

RESULT

The malware dataset of shape (10, 6) with 60 float64 values was prepared successfully. No missing values were found. The feature means were [789, 100.5, 6.69, 6.2, 284.5]. After Min-Max scaling all features lay between 0 and 1. Four files had entropy greater than 7, and the dataset contained 5 malware and 5 benign samples. After shuffling, the data was split into a training set of 8 samples (X_train 8 × 5) and a testing set of 2 samples (X_test 2 × 5).

CONCLUSION

NumPy slicing, aggregation, Boolean masking and random shuffling were used to transform raw malware analysis records into normalised training and testing arrays that are directly compatible with scikit-learn models.

Exp. No: 3.a	Supervised Learning Regression: Prediction of Detection time of Cyber incidents
Date: 28-07-2026	

AIM:

To design a supervised regression model that predicts the detection time of cyber incidents, and to report the model performance with its regression equation, evaluation metrics and visualisation plots.

OBJECTIVE

1. To design a supervised learning workflow for predicting detection time.
2. To fit a multiple linear regression model and formulate the regression equation.
3. To evaluate the model using MAE, MSE, RMSE and R^2 score.
4. To predict the detection time for new cyber incidents and identify the fastest and slowest detection.
5. To plot actual versus predicted values and comment on whether the model generalises well.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- NumPy, Pandas, Matplotlib, scikit-learn
- Dataset: Cyberattack.csv (15 records)

DESCRIPTION

Multiple linear regression models a continuous target as a linear combination of several input features: $Y = b_0 + b_1X_1 + b_2X_2 + \ldots + b_nX_n$, where b_0 is the intercept and $b_1 \ldots b_n$ are the coefficients estimated by ordinary least squares. In this experiment the target is **Detection_Time** (minutes) and the features are Traffic_MB, Failed_Logins, CPU_Usage and Network_Latency. The model is evaluated with:

- **MAE** – mean of the absolute errors $|y - \hat{y}|$.
- **MSE** – mean of the squared errors $(y - \hat{y})^2$.
- **RMSE** – square root of MSE, in the same unit as the target.
- **R^2 score** – proportion of the variance of y explained by the model (1.0 is a perfect fit).

PROCEDURE

1. Load Cyberattack.csv with Pandas and inspect it using head(), info(), describe() and isnull().sum().
2. Select the four features as X and Detection_Time as y, and split the data 80:20 (random_state=42).
3. Fit a LinearRegression model, predict the test data and compute MAE, MSE, RMSE and R^2.
4. Display the intercept and coefficients to form the regression equation.
5. Predict the detection time for new incidents, identify the fastest and slowest detection and plot actual versus predicted values.

CODE

28_JULY_26_Supervised_Learning_Regression.ipynb

In [7]: Step - 1 : Import the required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

In [8]: Step 2: Load Dataset

```
df = pd.read_csv("Cyberattack.csv")

df.head()
```

In [9]:

```
df.tail()
```

In [10]: Step - 3: Describe the data

```
df.info() #Information about data - data types
```

In [11]:

```
df.describe()  #Five number summary
```

In [12]:

```
df.isnull().sum()
```

In [13]: Step 4: Separate the features and labels

```
x = df[['Traffic_MB','Failed_Logins','CPU_Usage','Network_Latency']]
y = df['Detection_Time']
```

In [14]: Step 5: Split training and test sample sets.

```
x_train,x_test,y_train,y_test = train_test_split(
    x,
    y,
    test_size = 0.2,
    random_state = 42
)
```

In [15]: Step 6: Fit the Regression Model

```
model = LinearRegression()
```

In [16]:

```
model.fit(x_train, y_train) # Fit the model using feature training which is x_train, and target feature which is y_train
```

In [17]: Step 7: Perform Prediction on the test sample sets.

```
y_pred = model.predict(x_test)
print(y_pred)
```

In [18]: Step 8: Evaluate the performance of prediction across the test dataset

```
mae = mean_absolute_error(y_test,y_pred)
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)

r2 = r2_score(y_test,y_pred)

print("MAE = ",mae)
print("MSE = ",mse)
print("RMSE = ",rmse)
print("R2 Score = ",r2)
```

In [19]: Step 9: Compare the Actuals and Predicted Output

```
comparison = pd.DataFrame({
   'Actual' : y_test,
   'Predicted' : y_pred
})
print(comparison)
```

In [20]: Step 10: Plot the results

```
plt.figure(figsize=(8,5))

plt.scatter(y_test,
          y_pred)

plt.xlabel("Actual Detection Time")

plt.ylabel("Predicted Detection Time")

plt.title("Actual vs Predicted")

plt.grid(True)

plt.show()
```

In [21]: Step 11: Define the coefficients

```
coefficients = pd.DataFrame({
   "Feature" : x.columns ,
   "Coefficients" : model.coef_ ,
})
print(coefficients)
```

In [22]: Predict for a New Cyber Incident

```
new_data = np.array([[450,22,85,42]])
prediction = model.predict(new_data)
print("Predicted Detection Time = ",prediction[0])
```

In [23]: Predict for a new dataset

```
def predict_detection_time (traffic, failed_logins, cpu, latency):
   prediction = (-8.75
              + 0.051*traffic
              + 0.810*failed_logins
```

```
            + 0.320*cpu
            + 0.540*latency)
   return prediction
print(predict_detection_time (450, 22,85,42))
```

In [25]:

```
print("Intercept =", model.intercept_)
for feature, coef in zip (x.columns, model.coef_):
   print (feature, "=", coef)
```

In [26]: Task 1: Predict Detection Time

```
import pandas as pd
new_data = pd. DataFrame({
'Traffic_MB': [150, 300, 550, 720, 400],
'Failed_Logins': [5, 15, 26, 38, 18],
'CPU_Usage': [48, 70, 91, 98, 80],
'Network_Latency': [14, 30, 52, 72, 38]
})
print(new_data)
```

In [27]:

```
predictions = model.predict(new_data)
new_data["Predicted_Detection_Time"] = predictions
print(new_data)
```

In [29]: Task 2: Identify the Fastest Detection

```
fastest = new_data.loc[
new_data["Predicted_Detection_Time"].idxmin()
]
print (fastest)
```

In [30]: Task 3: Identify the Slowest Detection

```
slowest = new_data.loc[
new_data["Predicted_Detection_Time"].idxmax()
]
print(slowest)
```

In [31]: Task 4: Plot Actual vs Predicted

```
actual = [21, 48, 80, 99, 61]
```

In [32]:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,5))
plt.scatter(actual, predictions)

plt.plot([min(actual), max(actual)],
        [min(actual), max(actual)],
        'r--')

plt.xlabel("Actual Detection Time")
plt.ylabel("Predicted Detection Time")
plt.title("Actual vs Predicted")
plt.grid(True)

plt.show()
```

In [34]: Task 5: Comment on Whether the Model Generalizes Well

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np
actual = np.array([21,48,80,99,61])
```

```
predicted = np.array([22.3,46.8,79.4,101.2,60.5])
mae = mean_absolute_error (actual, predicted)
mse = mean_squared_error(actual, predicted)
rmse = np.sqrt(mse)
r2 = r2_score (actual, predicted)
print("MAE =", round (mae, 2))
print("MSE =", round(mse, 3))
print("RMSE =", round(rmse, 2))
print("R2 =", round (r2,3))
```

OUTPUT

[8]:

	Device_ID	Traffic_MB	Failed_Logins	CPU_Usage	Network_Latency	Detection_Time
0	1	120	5	45	12	18
1	2	350	18	82	35	54
2	3	80	2	30	8	12
3	4	600	28	95	55	85
4	5	250	10	60	20	34

[9]:

	Device_ID	Traffic_MB	Failed_Logins	CPU_Usage	Network_Latency	Detection_Time
10	11	420	20	84	40	60
11	12	650	32	96	62	90
12	13	200	8	52	18	28
13	14	540	25	90	50	76
14	15	90	2	32	9	13

JUPYTER NOTEBOOK OUTPUT – IN [8], [9]

[10]:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 6 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Device_ID        15 non-null     int64
 1   Traffic_MB       15 non-null     int64
 2   Failed_Logins    15 non-null     int64
 3   CPU_Usage        15 non-null     int64
 4   Network_Latency  15 non-null     int64
 5   Detection_Time   15 non-null     int64
dtypes: int64(6)
memory usage: 852.0 bytes
```

[11]:

	Device_ID	Traffic_MB	Failed_Logins	CPU_Usage	Network_Latency	Detection_Time
count	15.000000	15.000000	15.000000	15.00000	15.000000	15.000000
mean	8.000000	336.666667	15.466667	66.40000	32.000000	47.866667
std	4.472136	218.228801	11.363266	25.51974	20.976177	30.326478
min	1.000000	80.000000	2.000000	30.00000	8.000000	12.000000
25%	4.500000	135.000000	5.500000	42.50000	13.500000	19.000000
50%	8.000000	300.000000	14.000000	70.00000	28.000000	45.000000
75%	11.500000	520.000000	24.500000	89.00000	49.000000	74.000000
max	15.000000	700.000000	35.000000	97.00000	70.000000	96.000000

JUPYTER NOTEBOOK OUTPUT – IN [10], [11]

```
[12]: Device_ID          0
      Traffic_MB         0
      Failed_Logins      0
      CPU_Usage          0
      Network_Latency    0
      Detection_Time     0
      dtype: int64

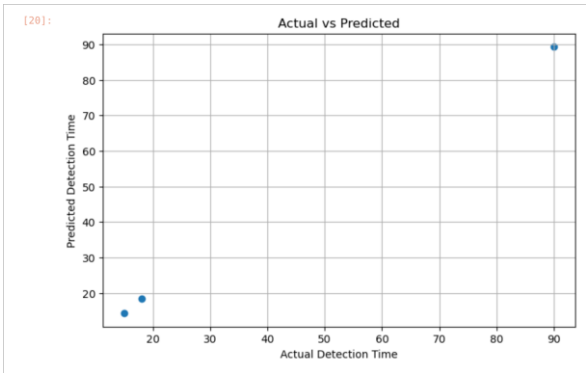
[16]: ▾ LinearRegression
      LinearRegression()

[17]: [14.30228078 89.2726977  18.47958766]

[18]: MAE =  0.6348697283058549
      MSE =  0.4152616920375465
      RMSE =  0.6444080167390428
      R2 Score =  0.9996545243826642

[19]:     Actual  Predicted
      9       15  14.302281
      11      90  89.272698
      0       18  18.479588
```

JUPYTER NOTEBOOK OUTPUT – IN [12], [16], [17], [18], [19]



ACTUAL VS PREDICTED (TEST SET)

```
[21]:          Feature  Coefficients
      0      Traffic_MB      0.085838
      1   Failed_Logins     -0.945059
      2       CPU_Usage      0.291211
      3  Network_Latency      0.719281

[22]: Predicted Detection Time =  63.966842594917885

[23]: 81.9

[25]: Intercept = -8.8314987384257
      Traffic_MB = 0.08583750350443736
      Failed_Logins = -0.9450592404382717
      CPU_Usage = 0.2912113046284771
      Network_Latency = 0.7192811226801951

[26]:    Traffic_MB  Failed_Logins  CPU_Usage  Network_Latency
      0         150              5         48               14
      1         300             15         70               30
      2         550             26         91               52
      3         720             38         98               72
      4         400             18         80               38
```

JUPYTER NOTEBOOK OUTPUT – IN [21], [22], [23], [25], [26]

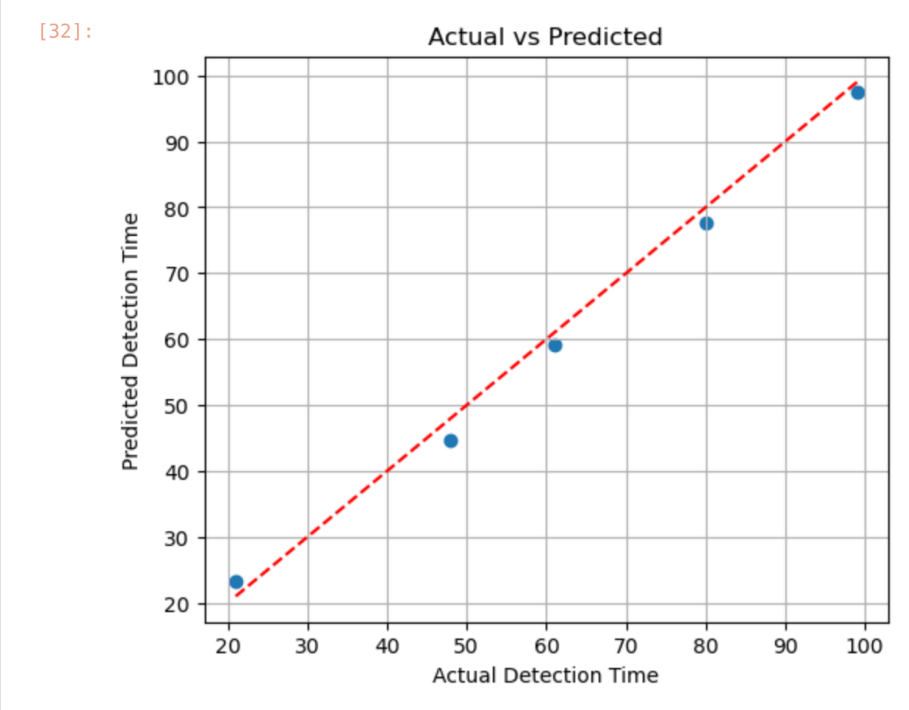
```
[27]:    Traffic_MB  Failed_Logins  CPU_Usage  Network_Latency  \
      0         150              5         48               14
      1         300             15         70               30
      2         550             26         91               52
      3         720             38         98               72
      4         400             18         80               38

         Predicted_Detection_Time
      0                 23.366909
      1                 44.707089
      2                 77.710435
      3                 97.386201
      4                 59.122023

[29]: Traffic_MB                 150.000000
      Failed_Logins                5.000000
      CPU_Usage                   48.000000
      Network_Latency             14.000000
      Predicted_Detection_Time    23.366909
      Name: 0, dtype: float64

[30]: Traffic_MB                 720.000000
      Failed_Logins               38.000000
      CPU_Usage                   98.000000
      Network_Latency             72.000000
      Predicted_Detection_Time    97.386201
      Name: 3, dtype: float64
```

JUPYTER NOTEBOOK OUTPUT – IN [27], [29], [30]



ACTUAL VS PREDICTED (NEW RECORDS)

```
[34]: MAE = 1.16
      MSE = 1.716
      RMSE = 1.31
      R2 = 0.998
```

TASK 5: COMMENT ON WHETHER THE MODEL GENERALIZES WELL

Regression equation (from the fitted model, In [18])

Detection_Time = −8.8315 + 0.0858 × Traffic_MB − 0.9451 × Failed_Logins + 0.2912 × CPU_Usage + 0.7193 × Network_Latency

Metric (test set, 3 samples)	Value
MAE	0.6349
MSE	0.4153
RMSE	0.6444
R^2 Score	0.9997

RESULT

The multiple linear regression model was trained on 12 records and tested on 3 records. On the test set it achieved MAE = 0.635, MSE = 0.415, RMSE = 0.644 and R^2 = 0.9997. For a new incident with 450 MB traffic, 22 failed logins, 85% CPU usage and 42 ms latency, the predicted detection time was **63.97 minutes**. Among the five new records, the fastest predicted detection was 23.37 minutes (Traffic 150 MB) and the slowest was 97.39 minutes (Traffic 720 MB). A manual calculation with rounded coefficients (In [17]) gave 81.9 minutes for the same incident. For Task 5, evaluation against the assumed actual detection times [21, 48, 80, 99, 61] gave MAE = 1.16, MSE = 1.716, RMSE = 1.31 and R^2 = 0.998.

CONCLUSION

A supervised regression workflow for predicting the detection time of cyber incidents was implemented. The model fits the available data very closely; however, because the dataset has only 15 records and the test set only 3, the very high R^2 should be interpreted with caution. Failed_Logins receives a negative coefficient even though it increases with detection time in the data, which indicates strong correlation among the features (multicollinearity). A larger dataset would be needed to confirm that the model generalises well.

Exp. No: 3.b	Investment on Advertising Media versus Sales
Date: 04-08-2026	

AIM:

To analyse the effect of investment in different advertising media (YouTube, Facebook and Newspaper) on sales using simple and multiple linear regression.

OBJECTIVE

1. To fit OLS regression models for each advertising medium, each pair and all three media.
2. To compare the models using R^2, adjusted R^2, F-statistic and p-values.
3. To visualise each medium against sales and study the YouTube × Facebook interaction.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- Pandas, statsmodels, seaborn, Matplotlib, scikit-learn, NumPy
- Dataset: 12 records of the "marketing" data (datarium package) entered in the notebook

DESCRIPTION

The **marketing** dataset records the advertising budget spent on YouTube, Facebook and Newspaper and the resulting **sales**. **statsmodels OLS** reports each coefficient with its standard error, t-value and p-value ($p < 0.05$ indicates a significant predictor), together with R^2 and adjusted R^2, which penalises predictors that do not improve the model. An **interaction term** (youtube × facebook) tests whether the effect of one medium depends on the other.

PROCEDURE

1. Create a DataFrame with the youtube, facebook, newspaper and sales columns.
2. Write a function run_model() that fits sm.OLS() with a constant and prints the summary.
3. Fit models for each medium, each pair of media and all three media.
4. Draw a pair plot and regression plots annotated with the fitted equations.
5. Fit a model with the interaction term youtube × facebook using the statsmodels formula API.

CODE

Exp_3b.ipynb

In [26]:

```
import pandas as pd
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
```

In [27]:

```
# Dataset from datarium (Marketing data)
data = {
   'youtube': [276.12, 53.4, 20.64, 23.72, 275.3, 55.12, 193.52, 2.41, 8.6, 219.68, 48.34, 178.72],
   'facebook': [45.36, 47.64, 55.04, 49.12, 43.4, 53.92, 54.64, 59.36, 61.52, 39.72, 44.28, 58.36],
   'newspaper': [83.04, 54.12, 83.04, 49.28, 80.64, 54.44, 41.52, 57.2, 64.0, 47.12, 55.92, 60.56],
   'sales': [26.52, 12.48, 11.16, 10.88, 26.36, 12.0, 20.16, 8.64, 9.6, 24.0, 11.2, 20.8]
}

df = pd.DataFrame(data)
df
```

In [28]:

```
# Create a user defined function to perform regression using OLS
# X's are features
# Y is the response or target
def run_model(features, response='sales'):
   """Fit and summarize an OLS model."""
   X = df[features]
   X = sm.add_constant(X)  # To calculate the intercept for the regression equation
   y = df[response]
   model = sm.OLS(y, X).fit()  # Fitting the OLS model from statsmodels library
   print(f"\nModel: {' + '.join(features)}")
   print(model.summary())
```

In [29]:

```
# 1. Each media individually
run_model(['youtube'])  # Call the function run_model(Advertising Media)
```

In [30]:

```
run_model(['facebook'])  # Call the function run_model(Advertising Media)
```

In [31]:

```
run_model(['newspaper'])  # Call the function run_model(Advertising Media)
```

In [32]:

```
# 2. Pairs of media
run_model(['youtube', 'facebook' ])
run_model(['youtube', 'newspaper' ])
run_model(['facebook', 'newspaper'])
```

In [33]:

```
run_model(['youtube', 'facebook', 'newspaper'])  # All three media together
```

In [34]:

```
sns.pairplot(df, kind='reg')
plt.suptitle("Advertising Media vs Sales", y=1.02)
plt.show()
```

In [35]:

```
# Set plot style
sns.set(style="whitegrid")
```

```
# Create subplots for each media with regression line
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# YouTube
sns.regplot(x='youtube', y='sales', data=df, ax=axes[0], color='red', ci=None)
axes[0]. set_title("YouTube Investment vs Sales")
axes[0].set_xlabel("YouTube Investment")
axes[0].set_ylabel("Sales")

# Facebook
sns.regplot(x='facebook', y='sales', data=df, ax=axes[1], color='blue', ci=None)
axes[1].set_title("Facebook Investment vs Sales")
axes[1]. set_xlabel("Facebook Investment")
axes[1].set_ylabel("Sales")

# Newspaper
sns.regplot(x='newspaper', y='sales', data=df, ax=axes[2], color='green', ci=None)
axes[2].set_title("Newspaper Investment vs Sales")
axes[2].set_xlabel("Newspaper Investment")
axes[2]. set_ylabel("Sales")

plt.tight_layout()
plt.show()
```

In [36]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
import numpy as np
# Set style
sns.set(style="whitegrid")

# Media channels
media = ['youtube', 'facebook', 'newspaper']
colors = ['red', 'blue', 'green']
titles = ['YouTube', 'Facebook', 'Newspaper']

# Create subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Loop through each media
for i, m in enumerate(media):
    X = df[[m]].values
    y = df['sales'].values
    model = LinearRegression()
    model.fit(X, y)
    slope = model.coef_[0]
    intercept = model.intercept_

    # Regression equation
    equation = f"Sales = {intercept:.2f} + {slope:.2f}x{m.capitalize()}"

    # Plot regression
    sns.regplot(x=m, y='sales', data=df, ax=axes[i], color=colors[i], ci=None)
    axes[i].set_title(f"{titles[i]} Investment vs Sales")
    axes[i].set_xlabel(f"{titles[i]} Investment")
    axes[i].set_ylabel("Sales")

    # Annotate equation
    axes[i].text(0.05, 0.95, equation,
```

```
        transform=axes[i].transAxes,
        fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='white', edgecolor='gray'))

plt.tight_layout()
plt.show()
```

In [37]:

```
# Create interaction term
df['youtube_facebook'] = df['youtube' ] * df['facebook']
```

In [38]:

```
# Build regression model with interaction term
import statsmodels.formula.api as smf # Import smf
model = smf.ols('sales ~ youtube + facebook + youtube_facebook', data=df).fit()

# Display summary
print(model.summary())
```

OUTPUT

[27]:

	youtube	facebook	newspaper	sales
0	276.12	45.36	83.04	26.52
1	53.40	47.64	54.12	12.48
2	20.64	55.04	83.04	11.16
3	23.72	49.12	49.28	10.88
4	275.30	43.40	80.64	26.36
5	55.12	53.92	54.44	12.00
6	193.52	54.64	41.52	20.16
7	2.41	59.36	57.20	8.64
8	8.60	61.52	64.00	9.60
9	219.68	39.72	47.12	24.00
10	48.34	44.28	55.92	11.20
11	178.72	58.36	60.56	20.80

JUPYTER NOTEBOOK OUTPUT – IN [27]

[29]:

```
Model: youtube
                            OLS Regression Results
==============================================================================
Dep. Variable:                  sales   R-squared:                       0.991
Model:                            OLS   Adj. R-squared:                  0.990
Method:                 Least Squares   F-statistic:                     1090.
Date:                Fri, 14 Aug 2026   Prob (F-statistic):           1.53e-11
Time:                        14:49:18   Log-Likelihood:                -11.419
No. Observations:                  12   AIC:                             26.84
Df Residuals:                      10   BIC:                             27.81
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          8.9337      0.295     30.280      0.000       8.276       9.591
youtube        0.0639      0.002     33.014      0.000       0.060       0.068
==============================================================================
Omnibus:                        0.093   Durbin-Watson:                   1.793
Prob(Omnibus):                  0.955   Jarque-Bera (JB):                0.316
Skew:                          -0.062   Prob(JB):                        0.854
Kurtosis:                       2.215   Cond. No.                         227.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

JUPYTER NOTEBOOK OUTPUT – IN [29]

[33]:

```
Model: youtube + facebook + newspaper
                            OLS Regression Results
==============================================================================
Dep. Variable:                  sales   R-squared:                       0.992
Model:                            OLS   Adj. R-squared:                  0.989
Method:                 Least Squares   F-statistic:                     325.7
Date:                Fri, 14 Aug 2026   Prob (F-statistic):           1.07e-08
Time:                        14:49:19   Log-Likelihood:                -10.742
No. Observations:                  12   AIC:                             29.48
Df Residuals:                       8   BIC:                             31.42
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          9.2713      2.139      4.335      0.002       4.340      14.203
youtube        0.0628      0.002     25.314      0.000       0.057       0.068
facebook      -0.0204      0.036     -0.559      0.591      -0.105       0.064
newspaper      0.0137      0.016      0.854      0.418      -0.023       0.050
==============================================================================
Omnibus:                        0.438   Durbin-Watson:                   2.263
Prob(Omnibus):                  0.804   Jarque-Bera (JB):                0.517
Skew:                           0.218   Prob(JB):                        0.772
Kurtosis:                       2.082   Cond. No.                     1.68e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.68e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

JUPYTER NOTEBOOK OUTPUT – IN [33]

[38]:

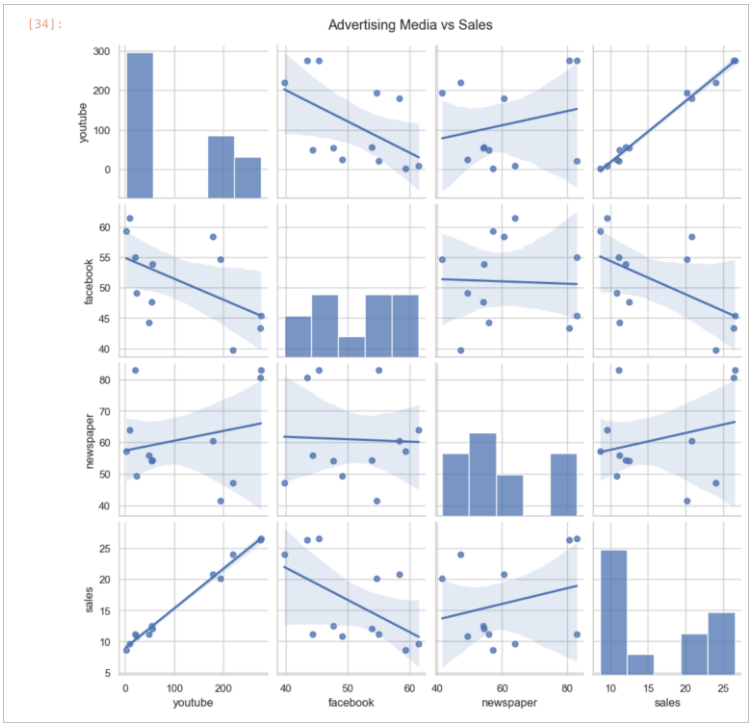
```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  sales   R-squared:                       0.992
Model:                            OLS   Adj. R-squared:                  0.989
Method:                 Least Squares   F-statistic:                     325.8
Date:                Fri, 14 Aug 2026   Prob (F-statistic):           1.07e-08
Time:                        14:49:26   Log-Likelihood:                -10.740
No. Observations:                  12   AIC:                             29.48
Df Residuals:                       8   BIC:                             31.42
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      8.1672      2.839      2.877      0.021       1.620      14.714
youtube        0.0774      0.017      4.636      0.002       0.039       0.116
facebook       0.0154      0.053      0.293      0.777      -0.106       0.137
youtube_facebook -0.0003      0.000     -0.856      0.417      -0.001       0.000
==============================================================================
Omnibus:                        2.282   Durbin-Watson:                   1.780
Prob(Omnibus):                  0.319   Jarque-Bera (JB):                0.989
Skew:                           0.210   Prob(JB):                        0.610
Kurtosis:                       1.658   Cond. No.                     9.75e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 9.75e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

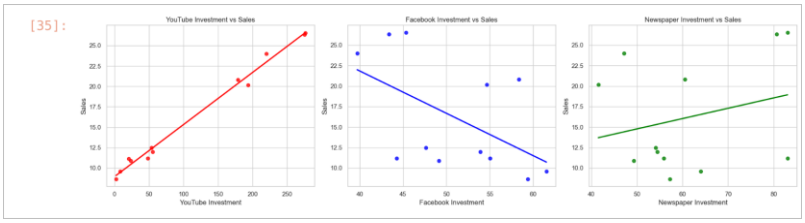
JUPYTER NOTEBOOK OUTPUT – IN [38]

Summary of all OLS models (values taken from the printed summaries)

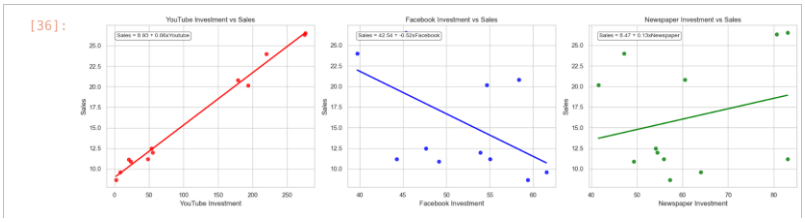
Model	R^2	Adj. R^2	F-statistic	Prob (F)
youtube	0.991	0.990	1090	1.53e-11
facebook	0.284	0.212	3.961	0.0746
newspaper	0.068	−0.026	0.7262	0.414
youtube + facebook	0.991	0.989	503.2	5.81e-10
youtube + newspaper	0.992	0.990	528.7	4.66e-10
facebook + newspaper	0.341	0.195	2.332	0.153
youtube + facebook + newspaper	0.992	0.989	325.7	1.07e-08
youtube + facebook + youtube×facebook	0.992	0.989	325.8	1.07e-08



PAIR PLOT – ADVERTISING MEDIA VS SALES



REGRESSION PLOTS OF EACH MEDIUM VS SALES



REGRESSION PLOTS WITH FITTED EQUATIONS

RESULT

YouTube alone explains 99.1% of the variation in sales ($R^2 = 0.991$, $p < 0.001$): Sales = 8.9337 + 0.0639 × YouTube. Facebook ($p = 0.075$) and Newspaper ($p = 0.414$) are not significant, adding them does not raise adjusted R^2 above 0.990, and the youtube × facebook interaction is not significant ($p = 0.417$).

CONCLUSION

For the 12 records analysed, YouTube advertising is the dominant and statistically significant driver of sales, while Facebook and Newspaper add little explanatory power.

Exp. No: 4.a	Decision Tree Classification
Date: 11-08-2026	

AIM:

To build and evaluate a Decision Tree classifier on the inbuilt Wine dataset of scikit-learn and to visualise the trained tree and its feature importances.

OBJECTIVE

1. To load the Wine dataset and split it into training and testing sets.
2. To train a Decision Tree classifier using the Gini criterion with a maximum depth of 4.
3. To evaluate the classifier using accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC.
4. To identify the most important features and visualise the decision tree.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- scikit-learn, NumPy, Matplotlib
- Dataset: Wine dataset (sklearn.datasets.load_wine)

DESCRIPTION

A **Decision Tree** is a supervised learning model that recursively splits the data on feature thresholds so that each child node becomes as pure as possible. With the **Gini** criterion, the impurity of a node is $\text{Gini} = 1 - \Sigma\, p_i^2$, where p_i is the proportion of class i in the node. Limiting max_depth controls model complexity and reduces overfitting. The **Wine dataset** contains 178 samples with 13 chemical features (alcohol, malic acid, flavanoids, colour intensity, proline, etc.) belonging to three cultivars (class_0, class_1, class_2). Feature importance in a tree is the total reduction in Gini impurity contributed by each feature.

PROCEDURE

1. Load the Wine dataset and split it 80:20 with stratification (random_state=42).
2. Train DecisionTreeClassifier(criterion="gini", max_depth=4, random_state=42).
3. Predict the test set and print the classification report and macro ROC-AUC score.
4. Plot the confusion matrix, ROC curves, feature importances and the trained tree.

CODE

Decision Tree Classification.ipynb

In [9]:

```
#1. Import necessary libraries
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
```

In [10]:

```
#2. Load the Inbuilt Wine Dataset
wine = load_wine()
X = wine.data
y = wine.target
```

In [13]:

```
# 3. Split the Data into Training and Testing Sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [14]:

```
# 4. Initialize and Train the Decision Tree Classifier
# max_depth is set to 4 to balance model complexity and performance
clf = DecisionTreeClassifier (criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)
```

In [15]:

```
#5. Make Predictions on the Test Set
y_pred = clf.predict(X_test)
```

In [16]:

```
import matplotlib.pyplot as plt
from sklearn.metrics import (
   ConfusionMatrixDisplay,
   RocCurveDisplay,
   classification_report,
   confusion_matrix,
   roc_auc_score,
)
from sklearn.preprocessing import LabelBinarizer

# Define target_names and n_classes
target_names = wine.target_names
n_classes = len(target_names)

# Generate Predictions and Probabilities
y_pred = clf.predict(X_test)
y_score = clf.predict_proba(X_test)  # Required for ROC curves

# ======================================================================
# PART A: CLASSIFICATION PERFORMANCE EVALUATION REPORT
# ======================================================================
print("============================================================")
print("         CLASSIFICATION PERFORMANCE REPORT             ")
print("============================================================")
print(classification_report(y_test, y_pred, target_names=target_names))

# Calculate Overall Macro AUC Score
```

```
# Binarize labels for multi-class AUC calculation
label_binarizer = LabelBinarizer().fit(y_train)
y_test_binarized = label_binarizer.transform(y_test)

macro_auc = roc_auc_score(
    y_test_binarized, y_score, multi_class="ovr", average="macro"
)
print(f"Overall Macro-averaged ROC AUC Score: {macro_auc:.4f}")
print("============================================================\n")


# ============================================================================
# PART B: VISUALIZATION (CONFUSION MATRIX & ROC CURVES)
# ============================================================================
# Create a single figure with side-by-side subplots
fig, ax = plt.subplots(1, 2, figsize=(14, 6))

# Plot 1: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
cm_display = ConfusionMatrixDisplay(
    confusion_matrix=cm, display_labels=target_names
)
cm_display.plot(ax=ax[0], cmap="Blues", values_format="d")
ax[0].set_title("Confusion Matrix Dashboard")
ax[0].grid(False)  # Remove grid lines inside the confusion matrix boxes

# Plot 2: One-vs-Rest Multi-class ROC Curves
colors = ["purple", "cyan", "darkorange"]

for class_idx in range(n_classes):
    RocCurveDisplay.from_predictions(
        y_test_binarized[:, class_idx],
        y_score[:, class_idx],
        name=f"ROC curve of class {target_names[class_idx]}",
        color=colors[class_idx % len(colors)],
        ax=ax[1],
        plot_chance_level=(
            class_idx == n_classes - 1
        ),  # Plot baseline once on the last iteration
    )

ax[1].set_xlabel("False Positive Rate (FPR)")
ax[1].set_ylabel("True Positive Rate (TPR)")
ax[1].set_title("Receiver Operating Characteristic (ROC) - Multi-Class")
ax[1].legend(loc="lower right")

plt.tight_layout()
plt.show()
```

In [17]:

```
# 5. Extract Feature Importances
importances = clf.feature_importances_
```

In [18]:

```
import numpy as np
# 6. Sort features by their importance in descending order
indices = np.argsort(importances) [::-1]
```

In [19]:

```
# Filter out features with 0 importance to keep the plot clean
import numpy as np
feature_names = np.array (wine.feature_names) # Convert to numpy array for ad
active_indices = [i for i in indices if importances [i] > 0]
sorted_importances = importances [active_indices]
sorted_features = feature_names [active_indices]
```

In [20]:

```
# # 7. Plot the Feature Importances
plt.figure(figsize=(10, 5))
bars = plt.barh(sorted_features[::-1], sorted_importances[::-1], color="teal")

# # Add text labels on top of the bars for exact scannability
for bar in bars:
    width = bar.get_width()
    plt.text(
        width + 0.01,
        bar.get_y() + bar.get_height()/2,
        f'{width:.2f}',
        va='center',
        ha='left',
        fontsize=10,
        fontweight='bold'
    )

plt.xlabel("Gini Importance Score")
plt.ylabel("Chemical Features")
plt.title("Decision Tree Feature Importance (Wine Dataset)")
plt.xlim(0, max(sorted_importances) + 0.1) # Extra space for text labels
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

In [21]:

```
#6. Evaluate the Model Performance
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%\n")
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=wine.target_names))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
#7. Visualize the Trained Decision Tree
plt.figure(figsize=(15, 10))
plot_tree(
clf,
feature_names=wine.feature_names,
class_names=wine.target_names,
filled=True,
rounded=True,
fontsize=10
)
plt.title("Visualized Decision Tree Classifier (Wine Dataset)")
plt.show()
```

OUTPUT

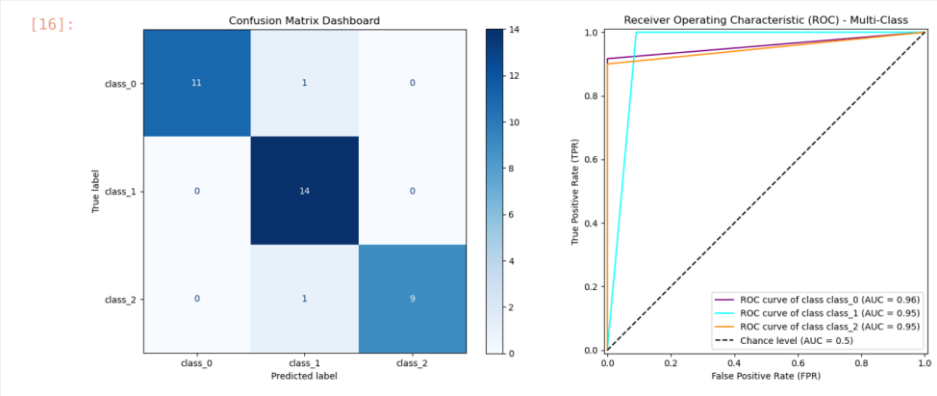
```
[16]: ========================================================
            CLASSIFICATION PERFORMANCE REPORT
      ========================================================
                    precision    recall  f1-score   support

           class_0       1.00      0.92      0.96        12
           class_1       0.88      1.00      0.93        14
           class_2       1.00      0.90      0.95        10

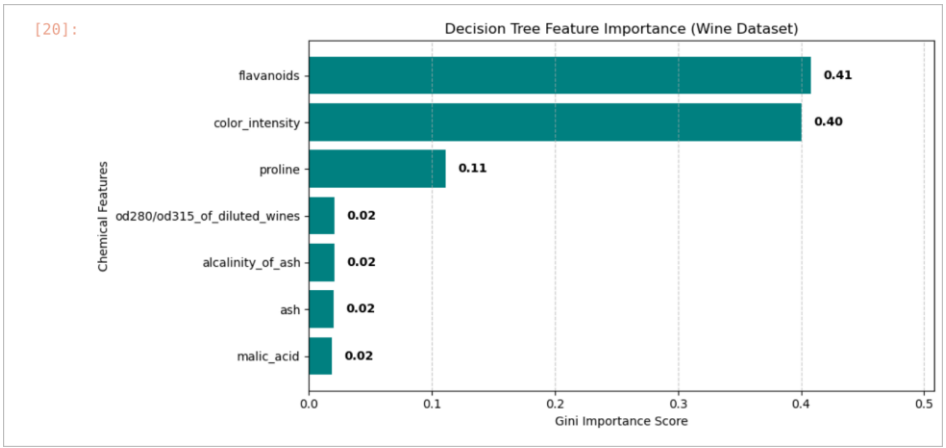
          accuracy                           0.94        36
         macro avg       0.96      0.94      0.95        36
      weighted avg       0.95      0.94      0.94        36

      Overall Macro-averaged ROC AUC Score: 0.9543
      ========================================================
```

JUPYTER NOTEBOOK OUTPUT – IN [16]



CONFUSION MATRIX AND MULTI-CLASS ROC CURVES



DECISION TREE FEATURE IMPORTANCE

```
[21]: Model Accuracy: 94.44%

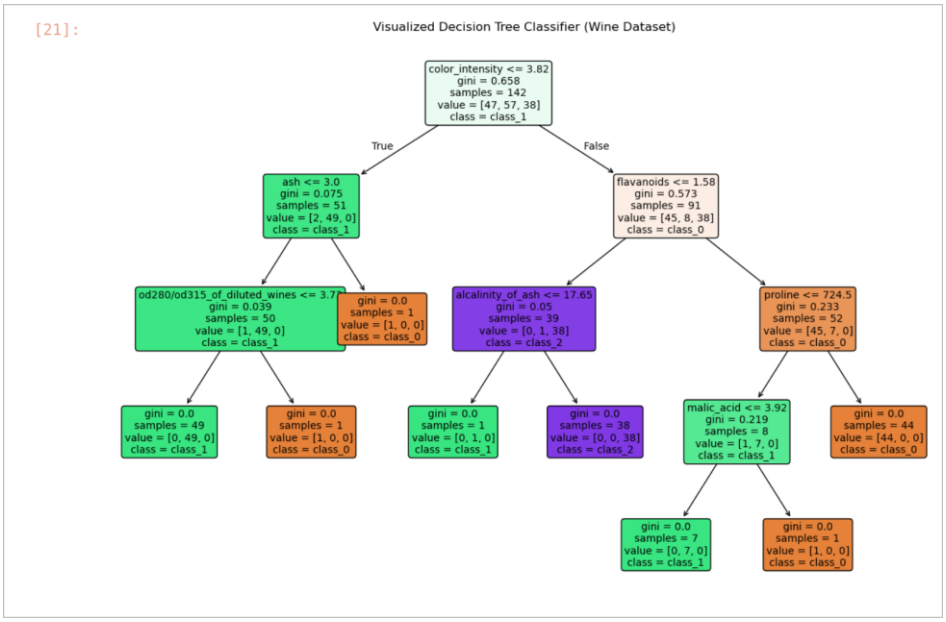
      Classification Report:
                    precision    recall  f1-score   support

           class_0       1.00      0.92      0.96        12
           class_1       0.88      1.00      0.93        14
           class_2       1.00      0.90      0.95        10

          accuracy                           0.94        36
         macro avg       0.96      0.94      0.95        36
      weighted avg       0.95      0.94      0.94        36

      Confusion Matrix:
      [[11  1  0]
       [ 0 14  0]
       [ 0  1  9]]
```

JUPYTER NOTEBOOK OUTPUT – IN [21]



VISUALISED DECISION TREE CLASSIFIER

RESULT

The Decision Tree classifier achieved an accuracy of **94.44%** on the 36 test samples, with a macro-averaged ROC-AUC of 0.9543. Two samples were misclassified (confusion matrix [[11 1 0], [0 14 0], [0 1 9]]). The most important features were flavanoids (0.41), color_intensity (0.40) and proline (0.11).

CONCLUSION

A depth-limited Decision Tree classified the three wine cultivars with high accuracy using only a few chemical features. The tree visualisation and feature-importance plot make the decision rules easy to interpret, which is a key advantage of decision trees.

Exp. No: 4.b.1	Supervised Learning Classification : Random Forest using Wine Quality dataset
Date: 18-08-2026	

AIM:

To implement a Random Forest classifier on the wine dataset and evaluate it using accuracy, a classification report, a confusion matrix and multi-class ROC curves.

OBJECTIVE

1. To load the wine dataset and split it into stratified training and testing sets.
2. To train a Random Forest with 100 decision trees.
3. To evaluate the predictions using accuracy and a classification report.
4. To plot the confusion matrix and one-vs-rest ROC curves for the three classes.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- scikit-learn, Matplotlib
- Dataset: Wine dataset (sklearn.datasets.load_wine)

DESCRIPTION

A **Random Forest** is an ensemble of decision trees. Each tree is trained on a bootstrap sample of the data and considers a random subset of features at every split (bagging + feature randomness). The final class is decided by majority voting, which reduces the variance and overfitting of individual trees. predict_proba() returns the fraction of trees voting for each class and is used to draw ROC curves. For a multi-class problem, the labels are binarised with LabelBinarizer and one ROC curve is plotted for each class against the rest.

PROCEDURE

1. Load the wine dataset and split it 80:20 with stratify=y and random_state=42.
2. Train RandomForestClassifier(n_estimators=100, random_state=42).
3. Predict the test labels and probabilities, and print the accuracy and classification report.
4. Plot the confusion matrix and the one-vs-rest ROC curves.

CODE

Random_Forest_Wine_dataset.ipynb

In [20]:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import(
   ConfusionMatrixDisplay,
   RocCurveDisplay,
   accuracy_score,
   classification_report,
   confusion_matrix,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer
```

In [21]:

```
wine = load_wine()
X,y = wine.data,wine.target
class_names = wine.target_names
```

In [30]:

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [27]:

```
rf_model = RandomForestClassifier(n_estimators=100,random_state=42)
```

In [28]:

```
rf_model.fit(X_train,y_train)
```

In [31]:

```
y_pred = rf_model.predict(X_test)
y_score = rf_model.predict_proba(X_test)
```

In [33]:

```
print("Accuracy: ", accuracy_score(y_test,y_pred))
print("Classification Report: \n", classification_report(y_test,y_pred))
```

In [36]:

```
cm = confusion_matrix(y_test, y_pred)
disp_cm = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)

fig,ax = plt.subplots(figsize=(6,6))
disp_cm.plot(ax=ax,cmap="Blues")
ax.set_title("Confusion Matrix")
plt.show()
```

In [41]:

```
label_binarizer = LabelBinarizer().fit(y_train)
y_onehot_test = label_binarizer.transform(y_test)

fig,ax = plt.subplots(figsize=(7,6))

for class_id,class_name in enumerate(class_names):
   RocCurveDisplay.from_predictions(
      y_onehot_test[:,class_id],
      y_score[:,class_id],
      name = f"{class_name} vs Rest",
      ax = ax
```

```
    )
ax.set_title("Extension of Reciever Operating Characteristic (ROC) to multiclass")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
plt.legend()
plt.show()
```

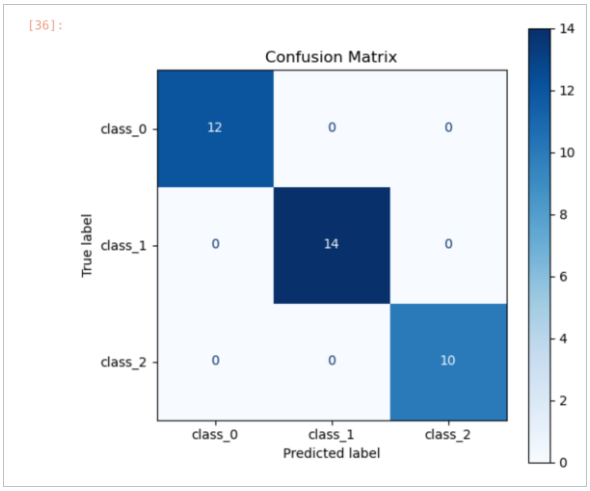
OUTPUT

```
[33]:  Accuracy:  1.0
       Classification Report:
                     precision    recall  f1-score   support

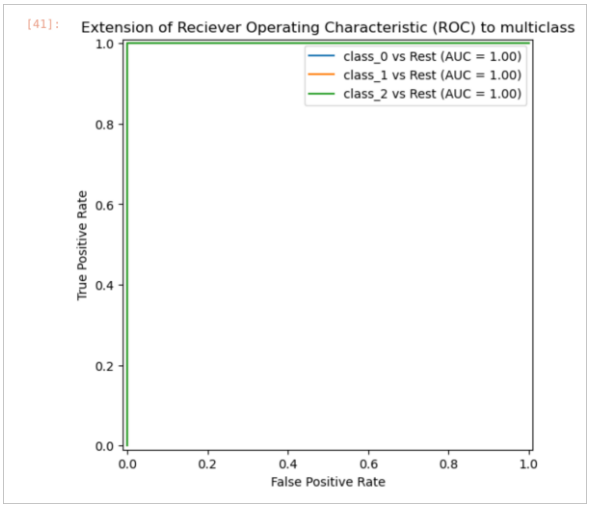
                  0       1.00      1.00      1.00        12
                  1       1.00      1.00      1.00        14
                  2       1.00      1.00      1.00        10

           accuracy                           1.00        36
          macro avg       1.00      1.00      1.00        36
       weighted avg       1.00      1.00      1.00        36
```

JUPYTER NOTEBOOK OUTPUT – IN [33]



CONFUSION MATRIX



MULTI-CLASS ROC CURVES

RESULT

The Random Forest classifier achieved an accuracy of **1.00 (100%)** on the 36 test samples; precision, recall and F1-score were 1.00 for all three classes, and the confusion matrix contained no misclassifications.

CONCLUSION

The ensemble of 100 trees classified every test sample correctly and outperformed the single Decision Tree of Experiment 4.a (94.44%) on the same train/test split, showing how bagging and random feature selection improve generalisation.

Exp. No: 4.b.2	Supervised Learning Classification : Random Forest using Digit dataset
Date: 18-08-2026	

AIM:

To implement a Random Forest classifier for recognising handwritten digits using the Digits dataset and to evaluate its performance.

OBJECTIVE

1. To load the Digits dataset and split it into stratified training and testing sets.
2. To train a Random Forest classifier with 100 trees.
3. To evaluate the model with accuracy and a per-class classification report.
4. To plot the confusion matrix and the ROC curve of each digit class.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- scikit-learn, Matplotlib
- Dataset: Digits dataset (sklearn.datasets.load_digits)

DESCRIPTION

The **Digits** dataset contains 1,797 8 × 8 grayscale images of handwritten digits (0–9), flattened into 64 pixel features. A Random Forest combines the votes of many decision trees trained on bootstrap samples and is evaluated with a per-class report, a 10 × 10 confusion matrix and one-vs-rest ROC curves.

PROCEDURE

1. Load the Digits dataset and split it 80:20 with stratify=y and random_state=42.
2. Train RandomForestClassifier(n_estimators=100, random_state=42).
3. Predict the test images and print the accuracy and classification report.
4. Plot the confusion matrix and the one-vs-rest ROC curves for the ten classes.

CODE

Random_Forest_Digit_dataset.ipynb

In [12]:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import(
   ConfusionMatrixDisplay,
```

```
    RocCurveDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer
```

In [13]:

```
digits = load_digits()
X = digits.data
y = digits.target
class_names = digits.target_names
```

In [14]:

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [15]:

```
rf_model = RandomForestClassifier(n_estimators=100,random_state=42)
```

In [16]:

```
rf_model.fit(X_train,y_train)
```

In [17]:

```
y_pred = rf_model.predict(X_test)
y_score = rf_model.predict_proba(X_test)
```

In [18]:

```
print("Accuracy: ", accuracy_score(y_test,y_pred))
print("Classification Report: \n", classification_report(y_test,y_pred))
```

In [19]:

```
cm = confusion_matrix(y_test, y_pred)
disp_cm = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)

fig,ax = plt.subplots(figsize=(6,6))
disp_cm.plot(ax=ax,cmap="Blues")
ax.set_title("Confusion Matrix")
plt.show()
```

In [20]:

```
label_binarizer = LabelBinarizer().fit(y_train)
y_onehot_test = label_binarizer.transform(y_test)

fig,ax = plt.subplots(figsize=(7,6))

for class_id,class_name in enumerate(class_names):
    RocCurveDisplay.from_predictions(
        y_onehot_test[:,class_id],
        y_score[:,class_id],
        name = f"{class_name} cs Rest",
        ax = ax
    )
ax.set_title("Extension of Reciever Operating Characteristic (ROC) to multiclass")
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
plt.legend()
plt.show()
```

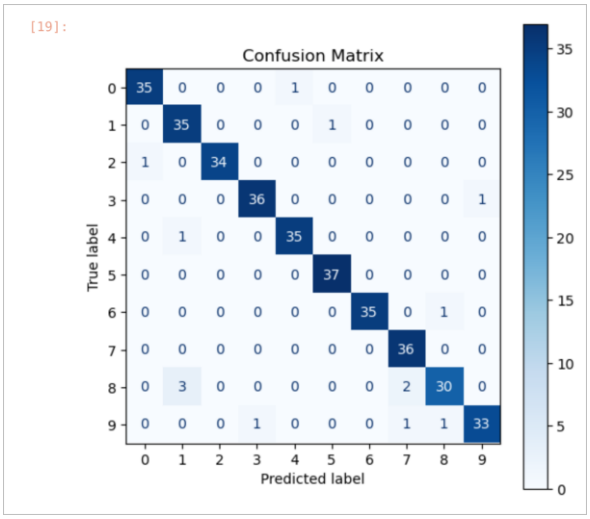
OUTPUT

```
[18]:  Accuracy:  0.9611111111111111
       Classification Report:
                     precision    recall  f1-score   support

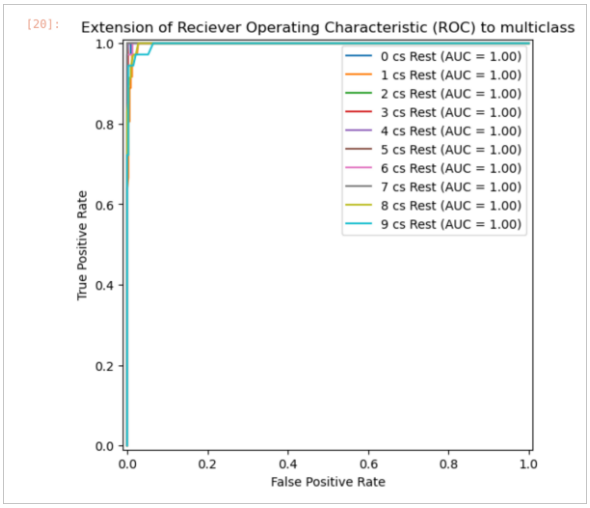
                  0       0.97      0.97      0.97        36
                  1       0.90      0.97      0.93        36
                  2       1.00      0.97      0.99        35
                  3       0.97      0.97      0.97        37
                  4       0.97      0.97      0.97        36
                  5       0.97      1.00      0.99        37
                  6       1.00      0.97      0.99        36
                  7       0.92      1.00      0.96        36
                  8       0.94      0.86      0.90        35
                  9       0.97      0.92      0.94        36

           accuracy                           0.96       360
          macro avg       0.96      0.96      0.96       360
       weighted avg       0.96      0.96      0.96       360
```

JUPYTER NOTEBOOK OUTPUT – IN [18]



CONFUSION MATRIX



MULTI-CLASS ROC CURVES

RESULT

The Random Forest classifier achieved an accuracy of **96.11%** on the 360 test images (macro F1-score 0.96). The lowest recall was observed for digit 8 (0.86) and the highest for digits 5 and 7 (1.00).

CONCLUSION

The Random Forest recognised handwritten digits with high accuracy directly from raw pixel features. The confusion matrix shows that most errors occur between visually similar digits, while the ROC curves confirm strong separability of every class.

Exp. No: 5.a	Estimate the Optimal Number of Clusters
Date: 25-08-2026	

AIM:

To estimate the optimal number of clusters for K-Means clustering using the Elbow method.

OBJECTIVE

1. To generate a synthetic dataset with a known cluster structure using make_blobs.
2. To fit K-Means for k = 1 to 10 and record the Within-Cluster Sum of Squares (WCSS).
3. To plot the Elbow curve and identify the optimal value of k.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- scikit-learn, Matplotlib

DESCRIPTION

K-Means partitions data into k clusters by repeatedly assigning each point to the nearest centroid and recomputing the centroids (k-means++ spreads the initial centroids apart). Since k must be chosen in advance, the **Elbow method** plots the Within-Cluster Sum of Squares (WCSS, inertia_) against k and takes the point where the decrease levels off — the "elbow" — as the optimal k.

PROCEDURE

1. Import Matplotlib, KMeans and make_blobs.
2. Generate 500 samples with 4 centres (cluster_std=0.60, random_state=42).
3. For each k from 1 to 10, fit KMeans(n_clusters=k, init="k-means++", n_init="auto") and store inertia_.
4. Plot WCSS against k and locate the elbow.

CODE

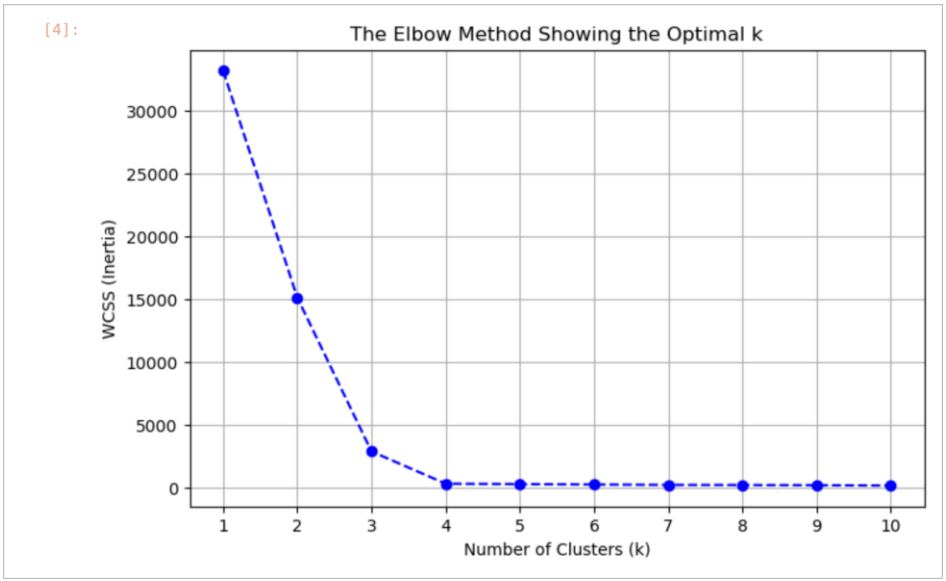
Optimal number of clusters.ipynb

In [4]:

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
# make_blobs imports a specific function from the scikit-learn (sklearn) library used to
#generate artificial datasets for testing clustering algorithms
# 1. Generate synthetic data with 4 distinct clusters
X, y = make_blobs (n_samples=500, centers=4, cluster_std=0.60, random_state=42)
```

```
# How to determine optimal number of clusters required for K-Means Clustering Algorithm.
# 2. Compute Within Cluster Sum of Squares WCSS (Inertia)
# for a range of k values
wcss = []
k_range = range (1, 11)
for k in k_range:
# 'n_init="auto"' sets initializations efficiently according to scikit-learn defaults
    kmeans = KMeans (n_clusters=k, init='k-means++', n_init='auto', random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_) # inertia is the WCSS value
#3. Plot the Elbow curve to find the optimal k
plt.figure(figsize=(8, 5))
plt.plot(k_range, wcss, marker='o', linestyle='--', color='b')
plt.title('The Elbow Method Showing the Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS (Inertia)')
plt.xticks(k_range)
plt.grid(True)
plt.show()
```

OUTPUT



ELBOW CURVE (WCSS VS NUMBER OF CLUSTERS)

RESULT

The WCSS dropped steeply from k = 1 to k = 4 and remained almost flat beyond k = 4. Therefore, the optimal number of clusters is **k = 4**, which matches the four centres used to generate the data.

CONCLUSION

The Elbow method correctly identified the number of natural groups in the data. It provides a simple, visual way to select k before applying K-Means to unlabelled data.

Exp. No: 5.b	Text Data Clustering using K means
Date: 25-08-2026	

AIM:

To cluster text documents into groups of similar topics using TF-IDF vectorisation and the K-Means algorithm, and to visualise the clusters in two dimensions using PCA.

OBJECTIVE

1. To convert raw text documents into TF-IDF feature vectors after removing English stop words.
2. To group the documents with K-Means (k = 2 and k = 3).
3. To reduce the TF-IDF vectors to two principal components and plot the clusters.
4. To study how the separation of topic vocabulary affects clustering quality.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- scikit-learn (TfidfVectorizer, KMeans, PCA), Pandas, seaborn, Matplotlib

DESCRIPTION

Clustering text requires converting documents into numbers. **TF-IDF** (Term Frequency – Inverse Document Frequency) gives a high weight to words that are frequent in a document but rare across the corpus: TF-IDF(t, d) = TF(t, d) × log(N / DF(t)). Each document becomes a sparse vector, and **K-Means** groups vectors that are close to each other. Because the vectors have thousands of dimensions, **Principal Component Analysis (PCA)** is used to project them onto two components for visualisation.

PROCEDURE

1. Define a list of short documents from different topics.
2. Vectorise the documents using TfidfVectorizer(stop_words="english").
3. Fit KMeans(n_clusters=k, init="k-means++", max_iter=200, n_init=10, random_state=42).
4. Print the cluster assigned to each document.
5. Reduce the TF-IDF matrix to two dimensions using PCA(n_components=2) and draw a labelled scatter plot.
6. Repeat with a larger 15-document corpus and with a high-separation corpus of three topics (culinary, finance, astronomy) using k = 3.

CODE

Text_Clustering_KMeans_E0224007.ipynb

In [2]:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
# 1. Sample Data (Already lightly preprocessed)
documents = [
    "Machine learning algorithms build models based on sample data.",
    "Data science and artificial intelligence are transforming industries.",
    "Deep neural networks require high-performance GPU computing.",
    "Homemade pasta dough requires flour, eggs, and water.",
    "The chef prepared a delicious three-course Italian dinner.",
    "Baking sourdough bread involves a long fermentation process."
]

#2. Vectorization: Convert text into a sparse TF-IDF matrix
# (stripping standard English stop words)
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(documents)

#3. K-Means: Initialize and fit the model ( K = 2 clusters)
# Using k-means++ for smarter initialization and setting a fixed random state
true_k = 2
model = KMeans (n_clusters=true_k, init='k-means++', max_iter=200, n_init=10, random_state=42)
model.fit(X)

#4. View Results.
for doc_idx, cluster_idx in enumerate (model.labels_):
    print (f"Cluster {cluster_idx}: {documents [doc_idx]}")
```

In [11]:

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.feature_extraction.text import TfidfVectorizer

# 1. Sample Data
documents = [
    "Machine learning algorithms build models based on sample data.",
    "Data science and artificial intelligence are transforming industries.",
    "Deep neural networks require high-performance GPU computing.",
    "Homemade pasta dough requires flour, eggs, and water.",
    "The chef prepared a delicious three-course Italian dinner.",
    "Baking sourdough bread involves a long fermentation process.",
]

# 2. Vectorization
vectorizer = TfidfVectorizer(stop_words="english")
X = vectorizer.fit_transform(documents)

# 3. K-Means Clustering
true_k = 2
model = KMeans(
    n_clusters=true_k, init="k-means++", max_iter=200, n_init=10, random_state=42
)
model.fit(X)
```

```
# 4. Dimensionality Reduction with PCA
pca = PCA(n_components=2, random_state=42)
pca_result = pca.fit_transform(X.toarray())

# Create a DataFrame for plotting
plot_df = pd.DataFrame(
    {
        "x": pca_result[:, 0],
        "y": pca_result[:, 1],
        "Cluster": model.labels_,
        "Document": documents,
    }
)

# Set the aesthetic style
sns.set_theme(style="whitegrid")

# Plot the clusters
plt.figure(figsize=(10, 6))
ax = sns.scatterplot(
    x="x",
    y="y",
    hue="Cluster",
    style="Cluster",
    palette="Set1",
    data=plot_df,
    s=150,
    alpha=0.8,
)

# Annotate the points
for i in range(len(plot_df)):
    label = plot_df["Document"][i]
    if len(label) > 40:
        label = label[:37] + "..."
    plt.text(
        plot_df["x"][i] + 0.02,
        plot_df["y"][i] + 0.02,
        label,
        fontsize=9,
        alpha=0.8,
    )

plt.title("Document Clusters Visualized in 2D Space (PCA)", fontsize=14, pad=15)
plt.xlabel("PCA Component 1", fontsize=12)
plt.ylabel("PCA Component 2", fontsize=12)
plt.xlim(plot_df["x"].min() - 0.2, plot_df["x"].max() + 0.6)
plt.ylim(plot_df["y"].min() - 0.2, plot_df["y"].max() + 0.2)
plt.legend(title="Cluster", loc="upper left")
plt.tight_layout()
plt.show()
```

In [17]:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
# 1. Expanded Sample Data (15 documents)
documents = [
    "Machine learning algorithms build models based on sample data.",
    "Data science and artificial intelligence are transforming industries.",
    "Deep neural networks require high-performance GPU computing.",
```

```
    "Natural language processing helps computers understand human language.",
    "Computer vision algorithms enable automated object detection in images.",

    "Homemade pasta dough requires flour, eggs, and water.",
    "The chef prepared a delicious three-course Italian dinner.",
    "Baking sourdough bread involves a long fermentation process.",
    "Slow-cooked beef stew seasoned with rosemary and thyme.",
    "Fresh basil and garlic create a classic Italian pesto sauce.",

    "Astronomers discovered a new planet orbiting a distant star.",
    "The James Webb Space Telescope captures high-resolution infrared images.",
    "Black holes exert immense gravitational forces that light cannot escape.",
    "Mars rover searches for evidence of past microbial life on the red planet.",
    "Saturn's rings are primarily composed of ice particles and space dust."
]

#2. Vectorization: Convert text into a sparse TF-IDF matrix
# (stripping standard English stop words)
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(documents)

#3. K-Means: Initialize and fit the model ( K = 2 clusters)
# Using k-means++ for smarter initialization and setting a fixed random state
true_k = 3
model = KMeans (n_clusters=true_k, init='k-means++', max_iter=200, n_init=10, random_state=42)
model.fit(X)

#4. View Results.
for doc_idx, cluster_idx in enumerate (model.labels_):
    print (f"Cluster {cluster_idx}: {documents [doc_idx]}")
```

In [15]:

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.feature_extraction.text import TfidfVectorizer

# 1. High-Separation Sample Data (15 documents)
documents = [
    # Category 1: Culinary / Baking (Flour, Dough, Oven, Recipe)
    "Preheat oven bake sourdough bread recipe flour yeast dough.",
    "Whisk eggs flour butter sugar bake chocolate cake recipe.",
    "Knead pasta dough flour water eggs olive oil sauce.",
    "Chef prepares grilled steak sauce garlic butter skillet recipe.",
    "Bake fresh pastry crust flour butter apples cinnamon dessert.",
    # Category 2: Finance & Stock Market (Wall Street, Dividends, Portfolio, Stocks)
    "Wall Street stock market equities investors trade shares exchange.",
    "Federal Reserve interest rates inflation central bank monetary policy.",
    "Investment portfolio diversification bonds stocks yields dividend returns.",
    "Quarterly earnings report stock prices corporate revenue profits margins.",
    "Cryptocurrency Bitcoin blockchain decentralized digital currency asset trading.",
    # Category 3: Astronomy & Deep Space (Galaxies, Telescopes, Orbit, Cosmic)
    "Telescope astronomers galaxy stars lightyears cosmic nebula space observation.",
    "Supernova explosion stellar collapse cosmic radiation space remnant galaxy.",
    "Orbit planet gravitational solar system asteroid comet atmosphere space.",
    "Astronauts space station microgravity orbit spacewalk spacecraft mission.",
    "Black hole event horizon gravity singularity spacetime astrophysics physics.",
]
```

```
# 2. Vectorization
vectorizer = TfidfVectorizer(stop_words="english")
X = vectorizer.fit_transform(documents)

# 3. K-Means Clustering (Set k=3 for the 3 distinct topics)
true_k = 3
model = KMeans(
    n_clusters=true_k, init="k-means++", max_iter=200, n_init=10, random_state=42
)
model.fit(X)

# 4. Dimensionality Reduction with PCA
pca = PCA(n_components=2, random_state=42)
pca_result = pca.fit_transform(X.toarray())

# Create a DataFrame for plotting
plot_df = pd.DataFrame(
    {
        "x": pca_result[:, 0],
        "y": pca_result[:, 1],
        "Cluster": model.labels_,
        "Document": documents,
    }
)

# Convert Cluster column to categorical so Seaborn treats it as discrete categories
plot_df["Cluster"] = plot_df["Cluster"].astype(str)

# 5. Plotting
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 8))

# Scatterplot with distinct color palette for 3 clusters
sns.scatterplot(
    x="x",
    y="y",
    hue="Cluster",
    style="Cluster",
    palette="Set1",
    data=plot_df,
    s=180,
    alpha=0.9,
)

# Annotate each point with shortened document text
for i in range(len(plot_df)):
    label = plot_df["Document"][i]
    if len(label) > 35:
        label = label[:32] + "..."
    plt.text(
        plot_df["x"][i] + 0.015,
        plot_df["y"][i] + 0.015,
        label,
        fontsize=8.5,
        alpha=0.85,
    )

plt.title("15 Document Clusters Visualized in 2D Space (PCA)", fontsize=14, pad=15)
plt.xlabel("PCA Component 1", fontsize=12)
plt.ylabel("PCA Component 2", fontsize=12)
```

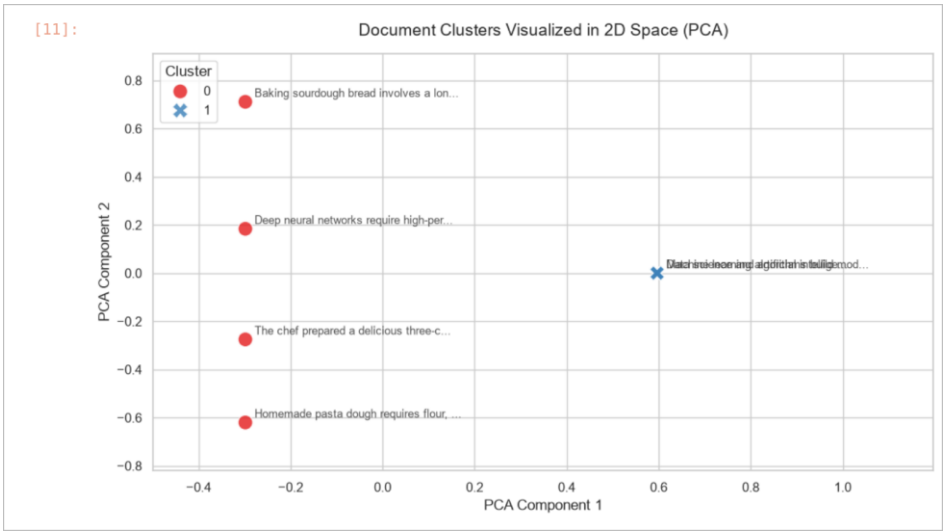
```
# Adjust plot boundaries to keep labels inside the frame
plt.xlim(plot_df["x"].min() - 0.1, plot_df["x"].max() + 0.35)
plt.ylim(plot_df["y"].min() - 0.1, plot_df["y"].max() + 0.15)

plt.legend(title="Cluster", loc="upper right")
plt.tight_layout()
plt.show()
```

OUTPUT

```
[2]: Cluster 1: Machine learning algorithms build models based on sample data.
     Cluster 1: Data science and artificial intelligence are transforming industries.
     Cluster 0: Deep neural networks require high-performance GPU computing.
     Cluster 0: Homemade pasta dough requires flour, eggs, and water.
     Cluster 0: The chef prepared a delicious three-course Italian dinner.
     Cluster 0: Baking sourdough bread involves a long fermentation process.
```

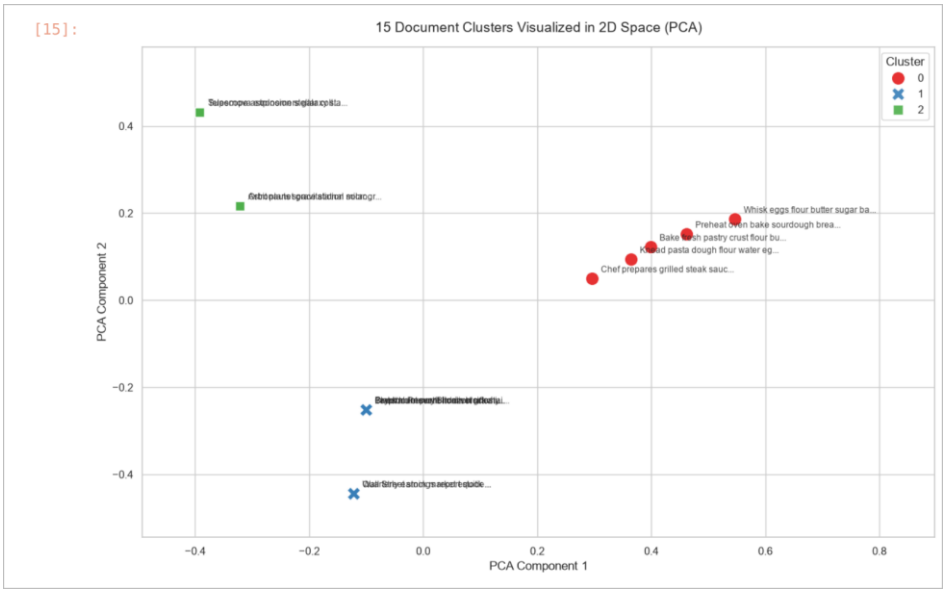
JUPYTER NOTEBOOK OUTPUT – IN [2]



DOCUMENT CLUSTERS (6 DOCUMENTS, K = 2)

```
[17]: Cluster 2: Machine learning algorithms build models based on sample data.
      Cluster 2: Data science and artificial intelligence are transforming industries.
      Cluster 1: Deep neural networks require high-performance GPU computing.
      Cluster 0: Natural language processing helps computers understand human language.
      Cluster 2: Computer vision algorithms enable automated object detection in images.
      Cluster 0: Homemade pasta dough requires flour, eggs, and water.
      Cluster 0: The chef prepared a delicious three-course Italian dinner.
      Cluster 0: Baking sourdough bread involves a long fermentation process.
      Cluster 0: Slow-cooked beef stew seasoned with rosemary and thyme.
      Cluster 0: Fresh basil and garlic create a classic Italian pesto sauce.
      Cluster 0: Astronomers discovered a new planet orbiting a distant star.
      Cluster 1: The James Webb Space Telescope captures high-resolution infrared images.
      Cluster 0: Black holes exert immense gravitational forces that light cannot escape.
      Cluster 0: Mars rover searches for evidence of past microbial life on the red planet.
      Cluster 1: Saturn's rings are primarily composed of ice particles and space dust.
```

JUPYTER NOTEBOOK OUTPUT – IN [17]



DOCUMENT CLUSTERS (15 DOCUMENTS, K = 3)

RESULT

With six documents and k = 2, the two machine learning/data science sentences formed one cluster while the remaining four (including the GPU sentence) formed the other. With the 15 mixed documents and k = 3, the clusters did not follow the intended topics, because the sentences share very few words. With the high-separation corpus, the PCA plot shows the five culinary documents grouped together in cluster 0 and the finance and astronomy documents placed in separate regions of the plot.

CONCLUSION

TF-IDF with K-Means can group documents by topic without labels, but the quality of the clusters depends strongly on the amount of shared, topic-specific vocabulary. Short sentences with little word overlap cluster poorly, while documents with distinctive keywords form clear, well-separated clusters.

Exp. No: 5.c	Association Rule Mining
Date: 01-09-2026	

AIM:

To discover frequent itemsets and association rules from a transactional dataset using the Apriori algorithm.

OBJECTIVE

1. To one-hot encode a list of market-basket transactions.
2. To find frequent itemsets using the Apriori algorithm with a minimum support threshold.
3. To generate association rules filtered by lift.
4. To compare the rules obtained with different support and lift thresholds.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- Pandas, mlxtend 0.25.0

DESCRIPTION

Association rule mining finds rules of the form A → B in transactional data. **Support** is the fraction of transactions containing both A and B, **confidence** = Support(A ∪ B) / Support(A), and **lift** = Confidence / Support(B), where lift > 1 means A and B occur together more often than by chance. The **Apriori** algorithm builds larger frequent itemsets only from smaller frequent ones, because every subset of a frequent itemset must also be frequent.

PROCEDURE

1. Install mlxtend and import Pandas, apriori, association_rules and TransactionEncoder.
2. Define five transactions containing Milk, Bread, Butter, Diapers and Beer.
3. One-hot encode the transactions with TransactionEncoder into a Boolean DataFrame.
4. Run apriori() with min_support=0.5 and generate rules with metric="lift", min_threshold=1.0.
5. Display antecedents, consequents, support, confidence and lift.
6. Repeat with min_support=0.3 and min_threshold=1.5.

CODE

Association Mining.ipynb

In [2]:
```
pip install mlxtend
```

In [1]:
```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
```

In [2]:
```
#1. Define sample transactional dataset
dataset = [
["Milk", "Bread", "Butter"],
["Bread", "Butter"],
["Milk", "Bread"],
["Milk", "Diapers", "Beer"],
["Milk", "Bread", "Butter", "Diapers", "Beer"],
]
```

In [3]:
```
#2. One-hot encode the transactional data
encoder = TransactionEncoder()
encoded_array =  encoder.fit(dataset).transform(dataset)
df = pd.DataFrame(encoded_array, columns=encoder.columns_)
```

In [4]:
```
#3. Find frequent itemsets using Apriori Algorithm
#min_support = 0.5 means items must appear in at least 50% of transactions
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)
```

In [5]:
```
#4. Generate association rules
#min_threshold 1.0 means we only keep rules where Lift >= 1
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)
```

In [6]:
```
#5. Clean up and display results
output_columns = [
   "antecedents",
   "consequents",
   "support",
   "confidence",
   "lift",
]
print(rules[output_columns].to_string())
```

In [42]:
```
#3. Find frequent itemsets using Apriori Algorithm
#min_support = 0.5 means items must appear in at least 50% of transactions
frequent_itemsets = apriori(df, min_support=0.3, use_colnames=True)
```

In [43]:
```
#4. Generate association rules
#min_threshold 1.0 means we only keep rules where Lift >= 1
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.5)
```

In [44]:
```
#5. Clean up and display results
output_columns = [
   "antecedents",
```

```
    "consequents",
    "support",
    "confidence",
    "lift",
]
print(rules[output_columns].to_string())
```

OUTPUT

[6]:

	antecedents	consequents	support	confidence	lift
0	frozenset({Bread})	frozenset({Butter})	0.6	0.75	1.25
1	frozenset({Butter})	frozenset({Bread})	0.6	1.00	1.25

[44]:

	antecedents	consequents	support	confidence	lift
0	frozenset({Beer})	frozenset({Diapers})	0.4	1.0	2.5
1	frozenset({Diapers})	frozenset({Beer})	0.4	1.0	2.5
2	frozenset({Beer, Milk})	frozenset({Diapers})	0.4	1.0	2.5
3	frozenset({Milk, Diapers})	frozenset({Beer})	0.4	1.0	2.5
4	frozenset({Beer})	frozenset({Milk, Diapers})	0.4	1.0	2.5
5	frozenset({Diapers})	frozenset({Beer, Milk})	0.4	1.0	2.5

JUPYTER NOTEBOOK OUTPUT – IN [6], [44]

RESULT

With support 0.5, two rules between Bread and Butter were found (lift 1.25). Lowering the support to 0.3 and raising the lift threshold to 1.5 revealed six strong rules linking Beer and Diapers, each with confidence 1.0 and lift 2.5.

CONCLUSION

The Apriori algorithm successfully extracted frequent itemsets and association rules from the transactions. The thresholds for support and lift control how many rules are produced and how strong they are; rules with high lift such as Beer → Diapers indicate items that are bought together far more often than by chance.

Exp. No: 6	The study of Reinforcement Learning Techniques
Date: 08-09-2026	

AIM:

To study reinforcement learning by implementing a Q-learning agent that learns to reach the goal in the FrozenLake environment of Gymnasium.

OBJECTIVE

1. To understand the agent–environment interaction loop of reinforcement learning.
2. To implement Q-learning with an epsilon-greedy policy and exponential epsilon decay.
3. To train the agent for 20,000 episodes and monitor the win rate.
4. To visualise the rewards, the rolling win rate and the learned policy.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- Gymnasium 1.3.0, pygame 2.6.1, NumPy, Matplotlib

DESCRIPTION

In **reinforcement learning** an agent learns by trial and error: in state s it takes action a, receives a reward r and moves to a new state s′. The goal is to learn a policy that maximises the total reward. **Q-learning** stores the expected return of every state–action pair in a Q-table and updates it with the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \, [\, r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \,]$$

where α is the learning rate and γ the discount rate. The **epsilon-greedy** strategy explores randomly with probability ε and otherwise exploits the best known action; ε decays from 1.0 towards 0.01 as ε = ε_min + (ε_max − ε_min)·e^(−decay·episode). **FrozenLake-v1** is a 4 × 4 grid world (16 states, 4 actions: left, down, right, up) in which the agent must walk from the start to the goal without falling into holes; reaching the goal gives a reward of 1. The non-slippery version (is_slippery=False) is used.

PROCEDURE

1. Install Gymnasium and pygame and create the FrozenLake-v1 environment (is_slippery=False).

2. Define a QLearningAgent class with a 16 × 4 Q-table, epsilon-greedy action selection, the Q-update and epsilon decay.
3. Train the agent for 20,000 episodes with $\alpha = 0.8$, $\gamma = 0.95$ and decay = 0.005.
4. Plot the rewards and the rolling win rate, then visualise the learned greedy policy.

CODE

Reinforcement_learning.ipynb

In [3]:

```
pip install gymnasium
```

In [4]:

```
pip install pygame
```

In [1]:

```
import random
import gymnasium as gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=False, render_mode="human")
env.reset()
```

In [2]:

```
import numpy as np
import gymnasium as gym

seed = 1773
np.random.seed(seed)

class QLearningAgent:
    def __init__(self, env, learning_rate, discount_rate, epsilon,
                 min_epsilon, max_epsilon, decay_rate):
        self.env = env
        self.learning_rate = learning_rate
        self.discount_rate = discount_rate
        self.epsilon = epsilon
        self.min_epsilon = min_epsilon
        self.max_epsilon = max_epsilon
        self.decay_rate = decay_rate
        self.q_table = np.zeros((env.observation_space.n, env.action_space.n))

    def choose_action(self, state):
        # Epsilon-greedy strategy
        if np.random.uniform(0, 1) < self.epsilon:
            action = self.env.action_space.sample()  # Explore
        else:
            action = np.argmax(self.q_table[state, :])  # Exploit
        return action

    def learn(self, state, action, reward, new_state, terminated=False):
        old_value = self.q_table[state, action]
        if terminated:
            next_max = 0.0
        else:
            next_max = np.max(self.q_table[new_state, :])

        new_q_value = old_value + self.learning_rate * (
            reward + self.discount_rate * next_max - old_value
```

```
        )
        self.q_table[state, action] = new_q_value

    def update_epsilon(self, episode):
        # Exponential decay of epsilon
        self.epsilon = self.min_epsilon + (
            self.max_epsilon - self.min_epsilon
        ) * np.exp(-self.decay_rate * episode)
```

In [3]:

```
learning_rate = 0.8
discount_rate = 0.95
max_epsilon = 1.0
min_epsilon = 0.01
decay_rate = 0.005
epsilon = max_epsilon

env = gym.make("FrozenLake-v1", is_slippery=False)

agent = QLearningAgent(
    env,
    learning_rate,
    discount_rate,
    epsilon,
    min_epsilon,
    max_epsilon,
    decay_rate
)
```

In [4]:

```
import matplotlib.pyplot as plt

render_env = gym.make("FrozenLake-v1", is_slippery=False, render_mode="rgb_array")
reset_result = render_env.reset(seed=seed)
state = reset_result[0] if isinstance(reset_result, tuple) else reset_result

frame = render_env.render()
plt.imshow(frame)
plt.axis("off")
plt.show()
render_env.close()
```

In [5]:

```
import time
from IPython.display import clear_output
import matplotlib.pyplot as plt

max_steps = 100
train_episodes = 20000

rewards_per_episode = np.zeros(train_episodes)
first_successful_episode = -1

print("Starting training...")

for episode in range(train_episodes):
    reset_result = env.reset()
    state = reset_result[0] if isinstance(reset_result, tuple) else reset_result
    total_episode_reward = 0
    terminated = False
    truncated = False
```

```
    for step in range(max_steps):
        action = agent.choose_action(state)
        new_state, reward, terminated, truncated, info = env.step(action)

        agent.learn(state, action, reward, new_state, terminated or truncated)
        total_episode_reward += reward
        state = new_state

        if terminated or truncated:
            break

    agent.update_epsilon(episode)
    rewards_per_episode[episode] = total_episode_reward

    if total_episode_reward == 1 and first_successful_episode == -1:
        first_successful_episode = episode + 1

    if (episode + 1) % 1000 == 0:
        current_win_rate = np.mean(rewards_per_episode[episode-999:episode+1] == 1)
        print(
            f"Episode {episode + 1}/{train_episodes}: "
            f"Win rate over last 1000 episodes: {current_win_rate:.2f}"
        )

print("\nTraining finished.")
print(f"First successful episode: {first_successful_episode}")
print(f"Average reward over last 100 episodes: {np.mean(rewards_per_episode[-100:]):.2f}")
```

In [6]:

```
# Plot training performance
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(rewards_per_episode)
plt.title("Rewards per Episode")
plt.xlabel("Episode")
plt.ylabel("Reward")

plt.subplot(1, 2, 2)
rolling_win_rate = np.convolve(
    rewards_per_episode == 1,
    np.ones(100) / 100,
    mode="valid"
)
plt.plot(rolling_win_rate)
plt.title("Rolling 100-Episode Win Rate")
plt.xlabel("Episode")
plt.ylabel("Win Rate")

plt.tight_layout()
plt.show()
```

In [7]: Visualization of Learned Policy

```
print("\nVisualizing a few episodes with the learned policy...")

agent.epsilon = 0.0
render_episodes = 5
visual_env = gym.make("FrozenLake-v1", is_slippery=False, render_mode="rgb_array")

for episode_viz in range(render_episodes):
```

```
    reset_result = visual_env.reset()
    state = reset_result[0] if isinstance(reset_result, tuple) else reset_result
    current_reward = 0

    time.sleep(0.5)

    for step in range(max_steps):
        clear_output(wait=True)

        frame = visual_env.render()
        plt.title(f"Visual Episode {episode_viz + 1}, Step {step + 1}")
        plt.imshow(frame)
        plt.axis("off")
        plt.show()

        action = agent.choose_action(state)
        new_state, reward, terminated, truncated, info = visual_env.step(action)
        current_reward = reward

        if terminated or truncated:
            clear_output(wait=True)
            final_frame = visual_env.render()
            title = (
                f"Visual Episode {episode_viz + 1}: Goal Reached!"
                if current_reward == 1
                else f"Visual Episode {episode_viz + 1}: Fell into a Hole!"
            )
            plt.title(title)
            plt.imshow(final_frame)
            plt.axis("off")
            plt.show()
            time.sleep(1)
            break

        state = new_state

visual_env.close()
env.close()
```

OUTPUT

```
[1]:  (0, {'prob': 1})
```

JUPYTER NOTEBOOK OUTPUT – IN [1]

[4]:



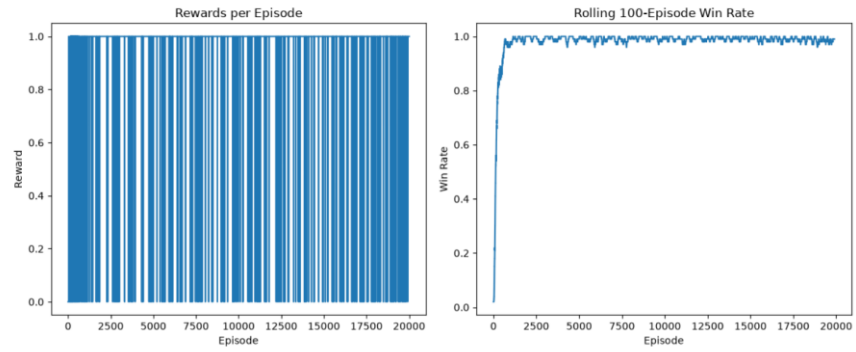
FROZENLAKE INITIAL STATE

```
[5]: Starting training...
Episode 1000/20000: Win rate over last 1000 episodes: 0.75
Episode 2000/20000: Win rate over last 1000 episodes: 0.99
Episode 3000/20000: Win rate over last 1000 episodes: 0.99
Episode 4000/20000: Win rate over last 1000 episodes: 0.99
Episode 5000/20000: Win rate over last 1000 episodes: 0.99
Episode 6000/20000: Win rate over last 1000 episodes: 0.99
Episode 7000/20000: Win rate over last 1000 episodes: 0.99
Episode 8000/20000: Win rate over last 1000 episodes: 0.99
Episode 9000/20000: Win rate over last 1000 episodes: 0.99
Episode 10000/20000: Win rate over last 1000 episodes: 0.99
Episode 11000/20000: Win rate over last 1000 episodes: 0.99
Episode 12000/20000: Win rate over last 1000 episodes: 0.99
Episode 13000/20000: Win rate over last 1000 episodes: 0.99
Episode 14000/20000: Win rate over last 1000 episodes: 0.99
Episode 15000/20000: Win rate over last 1000 episodes: 0.99
Episode 16000/20000: Win rate over last 1000 episodes: 0.99
Episode 17000/20000: Win rate over last 1000 episodes: 0.99
Episode 18000/20000: Win rate over last 1000 episodes: 0.99
Episode 19000/20000: Win rate over last 1000 episodes: 0.99
Episode 20000/20000: Win rate over last 1000 episodes: 0.98

Training finished.
First successful episode: 34
Average reward over last 100 episodes: 0.99
```

JUPYTER NOTEBOOK OUTPUT – IN [5]

[6]:



REWARDS PER EPISODE AND ROLLING WIN RATE



LEARNED POLICY – GOAL REACHED

RESULT

The Q-learning agent reached the goal for the first time in episode 34. Its win rate over the last 1,000 episodes rose to 0.75 after 1,000 episodes and to 0.99 from 2,000 episodes onwards; the average reward over the final 100 episodes was 0.99. Using the learned greedy policy ($\varepsilon = 0$), the final visualised episode (Visual Episode 5) reached the goal.

CONCLUSION

Q-learning with epsilon-greedy exploration learned a successful path to the goal in the FrozenLake grid purely from rewards, without any labelled data. The experiment illustrates the core ideas of reinforcement learning: states, actions, rewards, the exploration–exploitation trade-off and value updates using the Bellman equation.

Exp. No: 7.a	TF-IDF and Cosine Similarity for comparing two documents
Date: 15-09-2026	

AIM:

To represent text documents as numerical vectors using the Bag of Words and TF-IDF models, and to compare documents using cosine similarity.

OBJECTIVE

1. To pre-process text using regular expressions, tokenisation, stop-word removal and lemmatisation.
2. To build a Bag of Words (count) representation of documents.
3. To compute TF-IDF vectors after removing English stop words.
4. To measure the similarity between documents, and between a query and documents, using cosine similarity.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- NLTK 3.9.1, regex, scikit-learn (CountVectorizer, TfidfVectorizer, cosine_similarity), Pandas
- Input file: tamil_stopwords.txt (Tamil stop-word list)

DESCRIPTION

Before text can be compared, it must be cleaned and converted into numbers. **Pre-processing** removes punctuation, splits the text into tokens, removes stop words (very common words such as "the" and "is") and reduces each word to its dictionary form through **lemmatisation** (for example, mice → mouse). In the **Bag of Words** model, each document is represented as a vector of word counts over the vocabulary. **TF-IDF** weights each count according to how rare the word is across the documents, so that common words contribute less. **Cosine similarity** measures the angle between two vectors:

$$\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$$

Its value ranges from 0 (no common terms) to 1 (identical direction) and is independent of document length.

PROCEDURE

1. Remove punctuation using re.sub(r"[^\w\s]", "", text) and tokenise the text with word_tokenize and sent_tokenize.
2. Remove English stop words using NLTK and Tamil stop words using tamil_stopwords.txt.
3. Lemmatise the tokens with WordNetLemmatizer.
4. Create a Bag of Words matrix for three sentences using CountVectorizer and compute the cosine similarity matrix.
5. Create TF-IDF vectors for a three-document corpus and a query, and compute the cosine similarity between the query and each document.

CODE

Part A – Text pre-processing (NLP.ipynb)

In [2]:

```
import re
import string
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

In [3]:

```
nltk.download("stopwords")
nltk.download("punkt")
nltk.download("wordnet")
```

In [4]:

```
text = "Welcome to Natural Language Processing Session! We are now seeing an example senstence for N_L_P.
12_34"
print(text)
```

In [5]:

```
text = re.sub(r"[^\w\s]","",text)
#The regular expression ^\w\s is a combination to anchors and character classes used to match specific patterns
at the beginning of a string
# ^ : this is an anchor that asserts the position at the start of the string. It means the pattern must match from the
very beginning of the input text
# \w : This is a character class that matches any word character.
# This includes : Uppercase letters ( A-Z) Lowercase letters (a-z) Digits (0-9) Underscore(_)
# \s : This si a character class that matches any whitespace character.
# This includes: Space Tab (\t) Newline (\n) Carriage return (\r) Form feed (\f)
# Therefore, ^\w\s in a regular expression will match a string that starts with a single word character followed
immediately by a single whitespace character.
print(text)
```

In [6]:

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download("punkt_tab")
tokens = word_tokenize(text)
print(tokens)
```

In [13]:

```
import nltk
from nltk.tokenize import sent_tokenize , word_tokenize
```

```
nltk.download('punkt_tab')

text = "Dr. Smith went to the store. He bought some milk. He bought some protein powder. He came home and made a protein shake. He drank it and gained muscles"

tokens = word_tokenize(text)

print(sent_tokenize(text))
```

In [16]:

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words("english"))
token = [word for word in tokens if word.lower() not in stop_words]
print(token)
```

In [23]:

```
import regex as re

# 1. Path to your text file
stopwords_file = "tamil_stopwords.txt"

# Helper function to safely read stopwords
def load_stopwords(filepath):
    try:
        with open(filepath, "r", encoding="utf-8") as f:
            return set(line.strip() for line in f if line.strip())
    except FileNotFoundError:
        print(f"Error: Please ensure '{filepath}' is in the same directory.")
        return set()

# 2. Load the stop words
tamil_stopwords = load_stopwords(stopwords_file)

# 3. Sample text to filter
sample_text = "நான் சென்னைக்கு சென்றேன் ஆனால் அங்கே எனக்கு தங்குவதற்கு இடம் கிடைக்கவில்லை."
print(f"Original Text:\n{sample_text}\n")

# 4. Process the text using the 'regex' library with Unicode word boundaries
words = re.findall(r'\b\w+\b', sample_text, flags=re.UNICODE)

# 5. Filter words properly
filtered_words = [word for word in words if word not in tamil_stopwords]

# 6. Reconstruct using a space separator
cleaned_text = " ".join(filtered_words)

print(f"Filtered Word List:\n{filtered_words}\n")
print(f"Cleaned Text:\n{cleaned_text}")
```

In [26]:

```
import nltk
from nltk.stem import WordNetLemmatizer
#stemming uses crude, rule-based suffix femoval which often results in non-words
from nltk.tokenize import word_tokenize
nltk.download('wordnet')
nltk.download('punkt_tab')
text = "The mice and cats ate sweet candies while children played in different cities."
token = word_tokenize(text)
```

```
lemmatizer = WordNetLemmatizer()
token = [lemmatizer.lemmatize(word) for word in token]
print(token)
```

Part B – Bag of Words, TF-IDF and cosine similarity (Bag of Words.ipynb)

In [3]:

```
from sklearn.feature_extraction.text import CountVectorizer

#Define the texts
texts = [
    "Machine learning is amazing and fun",
    "Deep learning and machine learning are subsets of AI",
    "The weather outside is sunny and pleasant"
]

# Create the Bag of Words model
vectorizer = CountVectorizer()

# Fit and transform the texts into a matrix
X = vectorizer.fit_transform(texts)

# Print the vocabulary and vectors
print("Vocabuulary: " , vectorizer.get_feature_names_out())
print("\nVectors:\n", X.toarray())
```

In [5]:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

#Define the texts
texts = [
    "Machine learning is amazing and fun",
    "Deep learning and machine learning are subsets of AI",
    "The weather outside is sunny and pleasant"
]

# Create the Bag of Words model
vectorizer = CountVectorizer()

# Fit and transform the texts into a matrix
X = vectorizer.fit_transform(texts)

# Print the vocabulary and vectors
print("Vocabuulary: " , vectorizer.get_feature_names_out())
print("\nVectors:\n", X.toarray())

# Calculate Cosine Similarity
similarity_matrix = cosine_similarity(X)

# Print the results
print("Vocabuulary: " , vectorizer.get_feature_names_out())
print("\nCosine Similarity Matrix:\n", similarity_matrix)
```

In [12]:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# 1. Define a text corpus (documents)
corpus = [
```

```
    "Computer vision enables software to recognize and analyze visual data.",
    "Neural networks model complex patterns inspired by the human brain.",
    "Cloud computing provides scalable infrastructure for training large AI models."
]

# 2. Define a search query
query = ["How does Neural Networks and Computer vision work?"]

# 3. Combine corpus and query into a single list
all_texts = corpus + query
all_texts
```

In [13]:

```
# 4. Initialize TfidfVectorizer and transform the texts
vectorizer = TfidfVectorizer(stop_words = "english")
tfidf_matrix = vectorizer.fit_transform(all_texts)
print(tfidf_matrix)
```

In [14]:

```
# 5. Extract the query vector (last element) and corpus vector (all prior elements)
query_vector = tfidf_matrix[-1]
corpus_vectors = tfidf_matrix[:-1]
```

In [15]:

```
# 6. Compute cosine similarity between the query and each document
similarity_scores = cosine_similarity(query_vector,corpus_vectors).flatten()
similarity_scores
```

OUTPUT

Part A – Text pre-processing

```
[4]: Welcome to Natural Language Processing Session! We are now seeing an example senstence for
     N_L_P. 12_34

[5]: Welcome to Natural Language Processing Session We are now seeing an example senstence for N
     _L_P 12_34

[6]: ['Welcome', 'to', 'Natural', 'Language', 'Processing', 'Session', 'We', 'are', 'now', 'seei
     ng', 'an', 'example', 'senstence', 'for', 'N_L_P', '12_34']

[13]: ['Dr. Smith went to the store.', 'He bought some milk.', 'He bought some protein powder.',
      'He came home and made a protein shake.', 'He drank it and gained muscles']

[16]: ['Dr.', 'Smith', 'went', 'store', '.', 'bought', 'milk', '.', 'bought', 'protein', 'powde
      r', '.', 'came', 'home', 'made', 'protein', 'shake', '.', 'drank', 'gained', 'muscles']

[23]: Original Text:
      நான் சென்னைக்கு சென்றேன் ஆனால் அங்கே எனக்கு தங்குவதற்கு இடம் கிடைக்கவில்லை.

      Filtered Word List:
      ['சென்னைக்கு', 'சென்றேன்', 'எனக்கு', 'தங்குவதற்கு', 'இடம்', 'கிடைக்கவில்லை']

      Cleaned Text:
      சென்னைக்கு சென்றேன் எனக்கு தங்குவதற்கு இடம் கிடைக்கவில்லை

[26]: ['The', 'mouse', 'and', 'cat', 'ate', 'sweet', 'candy', 'while', 'child', 'played', 'in',
      'different', 'city', '.']
```

JUPYTER NOTEBOOK OUTPUT – IN [4], [5], [6], [13], [16], [23], [26]

Part B – Bag of Words, TF-IDF and cosine similarity

```
[3]: Vocabuulary:  ['ai' 'amazing' 'and' 'are' 'deep' 'fun' 'is' 'learning' 'machine' 'of'
      'outside' 'pleasant' 'subsets' 'sunny' 'the' 'weather']

     Vectors:
      [[0 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0]
      [1 0 1 1 1 0 0 2 1 1 0 0 1 0 0 0]
      [0 0 1 0 0 0 1 0 0 0 1 1 0 1 1 1]]

[5]: Vocabuulary:  ['ai' 'amazing' 'and' 'are' 'deep' 'fun' 'is' 'learning' 'machine' 'of'
      'outside' 'pleasant' 'subsets' 'sunny' 'the' 'weather']

     Vectors:
      [[0 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0]
      [1 0 1 1 1 0 0 2 1 1 0 0 1 0 0 0]
      [0 0 1 0 0 0 1 0 0 0 1 1 0 1 1 1]]
     Vocabuulary:  ['ai' 'amazing' 'and' 'are' 'deep' 'fun' 'is' 'learning' 'machine' 'of'
      'outside' 'pleasant' 'subsets' 'sunny' 'the' 'weather']

     Cosine Similarity Matrix:
      [[1.         0.49236596 0.3086067 ]
      [0.49236596 1.         0.11396058]
      [0.3086067  0.11396058 1.        ]]

[12]: ['Computer vision enables software to recognize and analyze visual data.',
       'Neural networks model complex patterns inspired by the human brain.',
       'Cloud computing provides scalable infrastructure for training large AI models.',
       'How does Neural Networks and Computer vision work?']
```

JUPYTER NOTEBOOK OUTPUT – IN [3], [5], [12]

```
[13]: <Compressed Sparse Row sparse matrix of dtype 'float64'
            with 31 stored elements and shape (4, 27)>
        Coords        Values
        (0, 5)        0.29294638531870676
        (0, 24)       0.29294638531870676
        (0, 9)        0.37156534433335936
        (0, 22)       0.37156534433335936
        (0, 20)       0.37156534433335936
        (0, 1)        0.37156534433335936
        (0, 25)       0.37156534433335936
        (0, 7)        0.37156534433335936
        (1, 17)       0.29294638531870676
        (1, 16)       0.29294638531870676
        (1, 14)       0.37156534433335936
        (1, 4)        0.37156534433335936
        (1, 18)       0.37156534433335936
        (1, 12)       0.37156534433335936
        (1, 10)       0.37156534433335936
        (1, 2)        0.37156534433335936
        (2, 3)        0.3333333333333333
        (2, 6)        0.3333333333333333
        (2, 19)       0.3333333333333333
        (2, 21)       0.3333333333333333
        (2, 11)       0.3333333333333333
        (2, 23)       0.3333333333333333
        (2, 13)       0.3333333333333333
        (2, 0)        0.3333333333333333
        (2, 15)       0.3333333333333333
        (3, 5)        0.3722248517590162
        (3, 24)       0.3722248517590162
        (3, 17)       0.3722248517590162
        (3, 16)       0.3722248517590162
        (3, 8)        0.47212002654617047
        (3, 26)       0.47212002654617047
```

JUPYTER NOTEBOOK OUTPUT – IN [13]

```
[15]: array([0.21808385, 0.21808385, 0.        ])
```

JUPYTER NOTEBOOK OUTPUT – IN [15]

RESULT

Text pre-processing removed punctuation and stop words and reduced words such as "mice", "cats" and "cities" to their base forms. Using Bag of Words, the two machine-learning sentences had a cosine similarity of 0.492, whereas the weather sentence had similarities of only 0.309 and 0.114 with them. For the query "How does Neural Networks and Computer vision work?", the TF-IDF cosine similarity was 0.218 with both the computer-vision and the neural-network documents and 0.0 with the cloud-computing document.

CONCLUSION

Text pre-processing followed by Bag of Words or TF-IDF vectorisation converts documents into vectors that can be compared numerically. Cosine similarity correctly identified related documents and ranked documents by their relevance to a query, which is the basis of search engines and document-matching systems.

Exp. No: 7.b	Case study on TF-IDF and Cosine Similarity : Tweets dataset (Kaggle)
Date: 15-09-2026	

AIM:

To apply TF-IDF vectorisation and cosine similarity to a set of short text documents in order to measure how similar the documents are to one another.

OBJECTIVE

1. To read text records from a file.
2. To convert the records into TF-IDF feature vectors after removing English stop words.
3. To compute the pairwise cosine similarity matrix of the records.
4. To interpret the similarity scores and identify the most similar pair of records.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- scikit-learn (TfidfVectorizer, cosine_similarity), Pandas
- Input file: data.csv (four short text records)
- Reference: Kaggle notebook "Tweets | NLP Cosine Similarity | TF-IDF" (natchaphonkamhaeng)

DESCRIPTION

Short texts such as tweets and posts can be compared by converting them into numerical feature vectors. **TF-IDF (Term Frequency – Inverse Document Frequency)** assigns each word in a document the weight:

$$\textbf{TF-IDF}(t, d) = \textbf{TF}(t, d) \times \textbf{IDF}(t)$$

where TF(t, d) is the frequency of term t in document d and IDF(t) is high for terms that appear in few documents. Each document therefore becomes a vector with one dimension for every word in the vocabulary; words that are frequent in one document but rare in the others receive high weights, and stop words are removed. **Cosine similarity** then compares two vectors by the cosine of the angle between them, $\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$. A value close to 1 indicates documents with similar important words, and 0 indicates that the documents share no weighted terms. The same technique is used in the referenced Kaggle case study to find similar tweets.

Input data: data.csv

Line	Text record
0	Being into sports is more than just exercising. It's a lifestyle that gives me energy, discipline, and a sense of achievement every day.
1	Playing the guitar and singing helps me express my feelings. It's like creating my own language through music that speaks to my heart.
2	I really like watching movies. Whether it's a funny one or a touching story, I enjoy the different feelings they bring.
3	Whether it's hitting the gym or doing outdoor activities, staying active and feeling that exercise buzz is a crucial part of my commitment to a healthy lifestyle.

PROCEDURE

1. Import Pandas, TfidfVectorizer and cosine_similarity.
2. Read data.csv line by line and extract lines 2 to 5 (the four text records), removing the trailing newline characters.
3. Create a TfidfVectorizer with English stop words removed, and fit and transform the records into a TF-IDF matrix.
4. Compute the cosine similarity between every pair of records.
5. Display the vocabulary and the cosine similarity matrix.

CODE

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 1. Read the text file directly line-by-line
with open('data.csv', 'r', encoding='utf-8') as f:
    lines = f.readlines()

# 2. Extract lines 2 to 5 and remove trailing newlines
text_data = [line.strip() for line in lines[1:5]]

# 3. Initialize and fit TfidfVectorizer
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(text_data)

# 4. Calculate Cosine Similarity
similarity_matrix = cosine_similarity(tfidf_matrix)

# 5. Display Results
print("Vocabulary:", vectorizer.get_feature_names_out())
print("\nCosine Similarity Matrix:\n", similarity_matrix)
```

OUTPUT

```
[7]: Vocabulary: ['achievement' 'active' 'activities' 'bring' 'buzz' 'commitment'
 'creating' 'crucial' 'day' 'different' 'discipline' 'doing' 'energy'
 'enjoy' 'exercise' 'exercising' 'express' 'feeling' 'feelings' 'funny'
 'gives' 'guitar' 'gym' 'healthy' 'heart' 'helps' 'hitting' 'just'
 'language' 'lifestyle' 'like' 'movies' 'music' 'outdoor' 'playing'
 'really' 'sense' 'singing' 'speaks' 'sports' 'staying' 'story' 'touching'
 'watching']

Cosine Similarity Matrix:
 [[1.         0.         0.         0.05429608]
 [0.         1.         0.11584404 0.        ]
 [0.         0.11584404 1.         0.        ]
 [0.05429608 0.         0.         1.        ]]
```

VOCABULARY AND COSINE SIMILARITY MATRIX

RESULT

After stop-word removal, the four records produced a vocabulary of 44 terms. The cosine similarity matrix shows that the guitar/music record (line 1) and the movies record (line 2) are the most similar pair, with a score of 0.116, because both contain the terms "feelings" and "like". The two exercise-related records (lines 0 and 3) have a similarity of 0.054, as they share the term "lifestyle". All other pairs have a similarity of 0, and each record has a similarity of 1 with itself.

CONCLUSION

TF-IDF converted the text records into weighted numerical vectors, and cosine similarity quantified how closely the records are related. The low similarity scores show that the records describe different interests, while shared meaningful words produce non-zero similarity. The same approach can be applied to large collections of tweets to find similar posts.

Exp. No: 8	Text Classification using Multinomial Naïve Bayes Algorithm
Date: 22-09-2026	

AIM:

To classify text documents into topics using TF-IDF features and the Multinomial Naïve Bayes algorithm.

OBJECTIVE

1. To load four categories of the 20 Newsgroups dataset.
2. To convert the documents into TF-IDF feature vectors.
3. To train a Multinomial Naïve Bayes classifier.
4. To evaluate the classifier using accuracy, a classification report and a confusion-matrix heat map.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook
- scikit-learn, NumPy, Matplotlib, seaborn
- Dataset: 20 Newsgroups (alt.atheism, soc.religion.christian, comp.graphics, sci.med)

DESCRIPTION

Naïve Bayes classifiers apply Bayes' theorem, $P(\text{class} \mid \text{document}) \propto P(\text{class}) \times \Pi\, P(\text{word}_i \mid \text{class})$, with the "naïve" assumption that words are conditionally independent given the class. The **Multinomial** variant models word counts or TF-IDF weights and is well suited to text classification. It is fast to train, works well with high-dimensional sparse data and gives a strong baseline for topic classification. The **20 Newsgroups** dataset contains about 18,000 newsgroup posts on 20 topics; four topics are used here.

PROCEDURE

1. Import the required libraries.
2. Load the training and testing subsets of the four selected categories using fetch_20newsgroups.
3. Fit TfidfVectorizer on the training text and transform both training and testing text.
4. Train MultinomialNB() on the TF-IDF training matrix.
5. Predict the test set and print the accuracy and classification report.
6. Plot the confusion matrix as a seaborn heat map.

CODE

TopicClassification_E0224007.ipynb

In [5]:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Load 4 selected categories
categories = ['alt.atheism','soc.religion.christian','comp.graphics','sci.med']

train_data = fetch_20newsgroups(
    subset='train',
    categories=categories,
    shuffle=True,
    random_state=42
)

test_data = fetch_20newsgroups(
    subset='test',
    categories=categories,
    shuffle=True,
    random_state=42
)

print(f"Training samples: {len(train_data.data)}")
print(f"Testing samples: {len(test_data.data)}")
print(f"Target classes: {train_data.target_names}")
```

In [6]:

```
# Initialize the Vectorizer
vectorizer = TfidfVectorizer()

# Fit and transform the training data,
# transform the test data
X_train = vectorizer.fit_transform(train_data.data)
X_test = vectorizer.transform(test_data.data)

y_train = train_data.target
y_test = test_data.target

print(f"TF-IDF Matrix Shape (Train): {X_train.shape}")
```

In [7]:

```
# Instantiate the Model
model = MultinomialNB()

# Train the Model
model.fit(X_train, y_train)
```

In [8]:

```
# Predict on test data
y_pred = model.predict(X_test)
```

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy Score: {accuracy:.4f}\n")

# Detailed Classification Report
print("Classification Report:")
print(
   classification_report(
      y_test,
      y_pred,
      target_names=test_data.target_names
   )
)
```

In [9]:

```
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(8, 6))

sns.heatmap(
   cm,
   annot=True,
   fmt='d',
   xticklabels=test_data.target_names,
   yticklabels=test_data.target_names,
   cmap='Blues',
   ax=ax
)

plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.show()
```

OUTPUT

```
[5]: Training samples: 2257
     Testing samples: 1502
     Target classes: ['alt.atheism', 'comp.graphics', 'sci.med', 'soc.religion.christian']

[6]: TF-IDF Matrix Shape (Train): (2257, 35788)

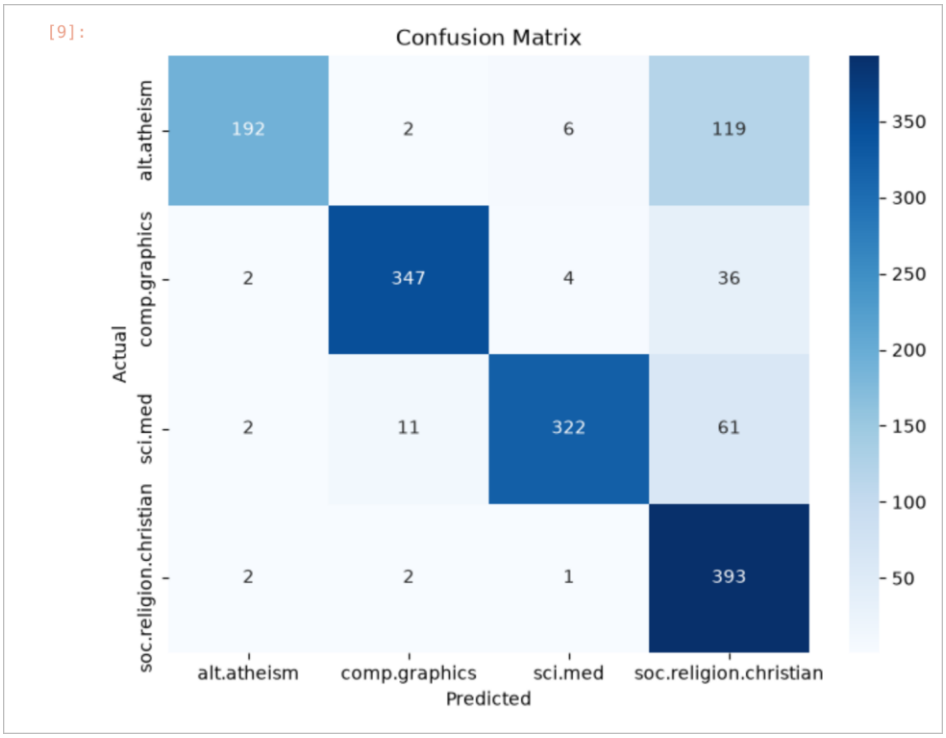
[8]: Accuracy Score: 0.8349

     Classification Report:
                            precision    recall  f1-score   support

                alt.atheism       0.97      0.60      0.74       319
              comp.graphics       0.96      0.89      0.92       389
                    sci.med       0.97      0.81      0.88       396
     soc.religion.christian       0.65      0.99      0.78       398

                   accuracy                           0.83      1502
                  macro avg       0.89      0.82      0.83      1502
               weighted avg       0.88      0.83      0.84      1502
```

JUPYTER NOTEBOOK OUTPUT – IN [5], [6], [8]



CONFUSION MATRIX

RESULT

The training set contained 2,257 documents and the test set 1,502 documents; the TF-IDF training matrix had 35,788 features. The Multinomial Naïve Bayes classifier achieved an accuracy of **83.49%** (weighted F1-score 0.84). comp.graphics had the highest F1-score (0.92), whereas many alt.atheism posts were predicted as soc.religion.christian (recall 0.60 for alt.atheism and 0.99 for soc.religion.christian).

CONCLUSION

TF-IDF features combined with Multinomial Naïve Bayes form a simple and effective text classifier. The main confusion occurred between the two closely related religion topics, which share much of their vocabulary.

Exp. No: 9	Machine Learning on AWS
Date: 29-09-2026	

AIM:

To implement and execute a Python data-processing notebook (Python Cheatsheet) on Amazon Web Services (AWS) using an Amazon SageMaker notebook instance.

OBJECTIVE

1. To access the Amazon SageMaker service from the AWS Management Console.
2. To open a Jupyter notebook environment running on a SageMaker notebook instance.
3. To upload and execute the Python Cheatsheet notebook in the cloud.
4. To install the required Python packages in the cloud environment using pip.
5. To perform NumPy, Pandas and Matplotlib operations in the cloud and observe the results.

AWS SERVICES AND ENVIRONMENT

Component	Details
Cloud platform	Amazon Web Services (AWS) – AWS Management Console
ML service	Amazon SageMaker – notebook instance with Jupyter
Compute	Notebook instance hosted on Amazon EC2 (user: ec2-user)
Environment	Anaconda environment "python3" at /home/ec2-user/anaconda3/envs/python3, Python 3.10
Libraries	NumPy 2.2.6, Pandas 2.3.3, Matplotlib 3.10.9, xlrd 2.0.2, openpyxl 3.1.5
Notebook	PythonCheatSheet.ipynb

DESCRIPTION

Amazon SageMaker is a fully managed AWS service for building, training and deploying machine learning models. A SageMaker **notebook instance** is a managed Amazon EC2 instance that runs Jupyter with pre-configured Anaconda environments, so code can be executed in the cloud without local installation. In this experiment, the **Python Cheatsheet** notebook (Python basics, NumPy, Pandas and plotting) is executed on the notebook instance to verify the cloud environment and practise data handling in the cloud.

PROCEDURE

1. Sign in to the AWS Management Console and open the **Amazon SageMaker** service.
2. Open **Notebook instances**, start the notebook instance and choose **Open Jupyter** when its status is **InService**.

3. Upload PythonCheatSheet.ipynb, open it and select the conda Python 3 kernel.
4. Install the required libraries with !pip install and run the NumPy, Pandas and plotting sections.
5. Save and download the executed notebook, and stop the notebook instance after use.

CODE

Representative cells from the notebook are given below.

PythonCheatSheet.ipynb (representative cells)

In [27]: 11. NumPy

```
# Install NumPy using pip
!pip install --upgrade pip
!pip install numpy
```

In [29]: Inspecting your array

```
# Create array
a = np.arange(15).reshape(3, 5) # Create array with range 0-14 in 3 by 5 dimension
b = np.zeros((3,5)) # Create array with zeroes
c = np.ones( (2,3,4), dtype=np.int16 ) # Createarray with ones and defining data types
d = np.ones((3,5))
```

In [33]:

```
a.size # Number of array elements
```

In [46]:

```
a.mean() # Array-wise mean
```

In [63]: Pandas

```
# Install pandas, xlrd, and openpyxl using pip
!pip install pandas
!pip install xlrd openpyxl
```

In [65]:

```
# Sample dataframe df
df = pd.DataFrame({'num_legs': [2, 4, np.nan, 0],
                   'num_wings': [2, 0, 0, 0],
                   'num_specimen_seen': [10, np.nan, 1, 8]},
                   index=['falcon', 'dog', 'spider', 'fish'])
df # Display dataframe df
```

In [72]:
df1.describe() # Display quick statistics summary of data

In [85]:
df.dropna(how='any') # Drop any rows that have missing data

In [87]:
df.fillna(value=5) # Fill missing data with value 5

In [90]:
df.to_csv('foo.csv') # Write to CSV file

In [91]:
pd.read_csv('foo.csv') # Read from CSV file

In [92]:
df.to_excel('foo.xlsx', sheet_name='Sheet1') # Write to Microsoft Excel file

In [93]:
pd.read_excel('foo.xlsx', 'Sheet1', index_col=None, na_values=['NA'], engine='openpyxl') # Read from Microsoft Excel file

In [96]:
Generate random time-series data
ts = pd.Series(np.random.randn(1000),index=pd.date_range('1/1/2000', periods=1000))
ts.head()

In [97]:
ts = ts.cumsum()
ts.plot() # Plot graph
plt.show()

OUTPUT

```
[27]: Requirement already satisfied: pip in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/
      site-packages (26.2)
      Collecting pip
        Downloading pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
      Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
         ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 74.5 MB/s  0:00:00
      Installing collected packages: pip
        Attempting uninstall: pip
          Found existing installation: pip 26.2
          Uninstalling pip-26.2:
            Successfully uninstalled pip-26.2
      Successfully installed pip-26.2.1
      Requirement already satisfied: numpy in /home/ec2-user/anaconda3/envs/python3/lib/python3.1
      0/site-packages (2.2.6)
```

PACKAGE INSTALLATION ON THE SAGEMAKER NOTEBOOK INSTANCE

```
[63]: Requirement already satisfied: pandas in /home/ec2-user/anaconda3/envs/python3/lib/python3.
      10/site-packages (2.3.3)
      Requirement already satisfied: numpy>=1.22.4 in /home/ec2-user/anaconda3/envs/python3/lib/p
      ython3.10/site-packages (from pandas) (2.2.6)
      Requirement already satisfied: python-dateutil>=2.8.2 in /home/ec2-user/anaconda3/envs/pyth
      on3/lib/python3.10/site-packages (from pandas) (2.9.0.post0)
      Requirement already satisfied: pytz>=2020.1 in /home/ec2-user/anaconda3/envs/python3/lib/py
      thon3.10/site-packages (from pandas) (2026.3.post1)
      Requirement already satisfied: tzdata>=2022.7 in /home/ec2-user/anaconda3/envs/python3/lib/
      python3.10/site-packages (from pandas) (2026.3)
      Requirement already satisfied: six>=1.5 in /home/ec2-user/anaconda3/envs/python3/lib/python
      3.10/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
      Collecting xlrd
        Downloading xlrd-2.0.2-py2.py3-none-any.whl.metadata (3.5 kB)
      Collecting openpyxl
        Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
      Collecting et-xmlfile (from openpyxl)
        Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
      Downloading xlrd-2.0.2-py2.py3-none-any.whl (96 kB)
      Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
      Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
      Installing collected packages: xlrd, et-xmlfile, openpyxl
         ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3/3 [openpyxl]2/3 [openpyxl]
      Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5 xlrd-2.0.2
```

PACKAGE INSTALLATION ON THE SAGEMAKER NOTEBOOK INSTANCE

[33]:
```
15
```

[46]:
```
np.float64(7.0)
```

[65]:

	num_legs	num_wings	num_specimen_seen
falcon	2.0	2	10.0
dog	4.0	0	NaN
spider	NaN	0	1.0
fish	0.0	0	8.0

[72]:

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	0.190243	0.035382	0.540046	-0.742253
std	0.328619	0.381025	0.587432	1.083388
min	-0.173731	-0.290485	-0.367162	-2.221937
25%	0.073222	-0.237996	0.209941	-1.088518
50%	0.103990	-0.141914	0.648773	-0.822328
75%	0.207821	0.326861	0.967647	-0.593464
max	0.801813	0.573509	1.172289	1.107496

JUPYTER NOTEBOOK OUTPUT – IN [33], [46], [65], [72]

[85]:

	num_legs	num_wings	num_specimen_seen
falcon	2.0	2	10.0
fish	0.0	0	8.0

[87]:

	num_legs	num_wings	num_specimen_seen
falcon	2.0	2	10.0
dog	4.0	0	5.0
spider	5.0	0	1.0
fish	0.0	0	8.0

[91]:

	Unnamed: 0	num_legs	num_wings	num_specimen_seen
0	falcon	2.0	2	10.0
1	dog	4.0	0	NaN
2	spider	NaN	0	1.0
3	fish	0.0	0	8.0

JUPYTER NOTEBOOK OUTPUT – IN [85], [87], [91]

[93]:

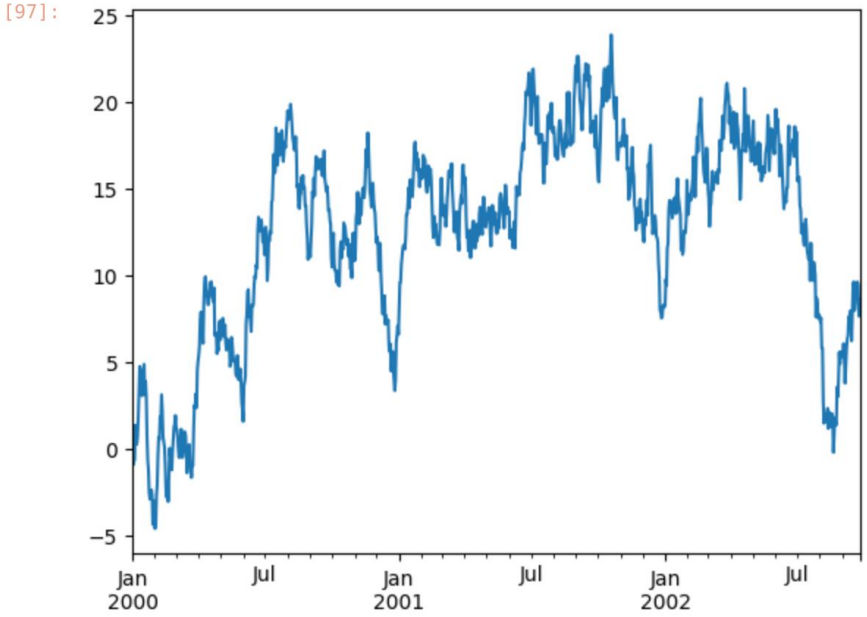
	Unnamed: 0	num_legs	num_wings	num_specimen_seen
0	falcon	2.0	2	10.0
1	dog	4.0	0	NaN
2	spider	NaN	0	1.0
3	fish	0.0	0	8.0

[96]:

```
2000-01-01   -0.714224
2000-01-02   -0.208858
2000-01-03    0.252314
2000-01-04    0.023377
2000-01-05    2.020994
Freq: D, dtype: float64
```

JUPYTER NOTEBOOK OUTPUT – IN [93], [96]

[97]:



TIME-SERIES PLOT GENERATED ON THE SAGEMAKER INSTANCE

OBSERVATIONS

- The installation logs show that the notebook ran in the environment /home/ec2-user/anaconda3/envs/python3 with Python 3.10 on the SageMaker notebook instance.
- NumPy 2.2.6 and Pandas 2.3.3 were already available in the environment; pip was upgraded to 26.2.1, and xlrd 2.0.2, openpyxl 3.1.5 and Matplotlib 3.10.9 were installed from within the notebook.
- Files written by the notebook (foo.csv, foo.xlsx) were stored on the instance and read back successfully.
- Plots were rendered directly in the cloud-hosted notebook.

RESULT

The Python Cheatsheet notebook was executed successfully on an Amazon SageMaker notebook instance. The required libraries were installed in the cloud environment, NumPy

operations returned the expected values (for example, the mean of np.arange(15) is 7.0), Pandas operations such as describe(), dropna() and fillna() and CSV file input/output worked correctly, and a time-series plot was generated in the cloud.

CONCLUSION

Amazon SageMaker provides a ready-to-use, cloud-hosted Jupyter environment for machine learning. Code can be executed and libraries installed on demand without any local setup, and the same environment can be used for training and deploying machine learning models on AWS.

Exp. No: 10.a	Serialization of ML models
Date: 06-10-2026	

AIM:

To serialise (save) trained machine learning models using pickle and joblib, to publish a serialised model in a GitHub repository, and to deserialise (load) and reuse the models for prediction.

OBJECTIVE

1. To train a linear regression model of salary versus years of experience and save it with pickle.dump().
2. To load the saved model with pickle.load() and predict salaries for new inputs.
3. To train a TF-IDF + Multinomial Naïve Bayes pipeline on the 20 Newsgroups dataset and save it with joblib.dump().
4. To host the trained model in the GitHub repository https://github.com/Swaminathan8237/MLCS.
5. To reuse the model by loading it directly from the repository and classifying new text samples.

SOFTWARE REQUIREMENTS

- Windows 10/11
- Anaconda Distribution (Python 3), Jupyter Notebook, Visual Studio Code
- NumPy, scikit-learn, pickle (standard library), joblib, requests
- Git and GitHub

DESCRIPTION

Serialization converts a trained model in memory into a byte stream that can be stored in a file; **deserialization** rebuilds the model from that file, so it can be reused without retraining. Python's built-in **pickle** module (pickle.dump() / pickle.load()) and the **joblib** library (joblib.dump() / joblib.load()), which is optimised for models containing large NumPy arrays, are used. A serialised model committed to **GitHub** can be loaded remotely by downloading the raw file with requests.get() and reading it through io.BytesIO. Models should be loaded only from trusted sources, because deserialising a pickle file can execute arbitrary code.

GitHub repository: https://github.com/Swaminathan8237/MLCS

File	Purpose
train.py	Loads the 20 Newsgroups dataset, trains the TF-IDF + Multinomial Naïve Bayes pipeline, evaluates it and saves it as 20newsgroups_model.joblib
20newsgroups_model.joblib	Serialised trained pipeline (about 33.9 MB)
requirements.txt	Python package dependencies
README.md	Project description and list of files

PROCEDURE

1. Fit a LinearRegression model on years of experience versus salary and save it to linear_model.pkl with pickle.dump().
2. Load the model with pickle.load() and predict the salary for 2.5 and 6 years of experience.
3. Write train.py to train a TF-IDF + MultinomialNB pipeline on 20 Newsgroups and save it as 20newsgroups_model.joblib.
4. Push train.py, requirements.txt, README.md and the model file to the GitHub repository.
5. Run predict.py to download the model from GitHub, load it with joblib.load() and classify new text.

CODE

Part A – Serialization of ML Model.ipynb

In [7]: Import the library

```
import numpy as np
import pickle
from sklearn.linear_model import LinearRegression
```

In [8]: Dataset

```
X = np.array([[4],[8],[15],[17],[22]])
y = np.array([[20],[45],[75],[80],[100]])
```

In [11]:

```
X
```

In [12]:

```
y
```

In [13]: Fit the Linear Regression Model

```
model = LinearRegression()
model.fit(X,y)
```

In [16]:

```
print("Model Parameters")
print(f"Intercept : {model.intercept_[0]:.2f}" )
print(f"Coefficient:{model.coef_[0][0]:.2f}")
```

In [19]: Serialization

```
model_filename = "D:/E0224033/ML_for_Cl_ser/ML_Cloud_project/linear_model.pkl"
with open(model_filename,"wb") as file:
   pickle.dump(model,file)
print("Model serialized and saved")
```

In [20]: Reuse the Model for Prediction

```
import numpy as np
import pickle
```

In [21]: Deserialization: Load the saved model back into memory

```
model_filename = " D:/E0224033/ML_for_Cl_ser/ML_Cloud_project/linear_model.pkl"
with open(model_filename,"rb") as file:
   loaded_model = pickle.load(file)
print("Model loaded successfully")
```

In [23]: Prepare the data sample for prediction

```
new_data = np.array([[2.5],[6]])
```

In [24]: Make prediction with pkl file

```
predictions = loaded_model.predict(new_data)
```

In [30]: Display Results

```
for exp, pred_salary in zip(new_data.flatten(), predictions):
   print(f"Predicted Salary for {exp} years of experience : ${pred_salary[0]:.2f}k")
```

Part B – train.py (GitHub repository)

```
import joblib
from pathlib import Path
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, classification_report
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

# Step 1: Load the 20 Newsgroups dataset (train and test sets)
print('Loading 20 Newsgroups dataset...')
train = fetch_20newsgroups (subset='train', remove = ('header', 'footers', 'quotes'))
test = fetch_20newsgroups (subset='test', remove = ('header', 'footers', 'quotes'))

# Step 2: Create a model pipeline (TF-IDF Vectorizer + Multinomial Naive Bayes)
print('Building the Multinomial Naive Bayes model pipeline...')
model = make_pipeline (TfidfVectorizer(), MultinomialNB())
# Step 3: Train the model
print('Training the model...')
model.fit(train.data, train.target)

# Step 4: Evaluate the model
predicted = model.predict(test.data)
acc = accuracy_score (test.target, predicted)
print(f'Test Accuracy: {acc * 100:.2f}%')
print('\nClassification Report:')
print(classification_report (test.target, predicted, target_names=test.target_names))

# Step 5: Save the model to disk
model_filename = Path('20newsgroups_model.joblib')
joblib.dump(model, model_filename)
print(f'Model saved successfully as {model_filename}')
```

Part B – requirements.txt (GitHub repository)

```
scikit-learn>=1.0.0
numpy>=1.20.0
joblib>=1.0.0
```

Part B – predict.py (model reuse)

```
import io

import joblib
import requests

# Define the raw GitHub URL of the saved model.
MODEL_URL = (
    "https://raw.githubusercontent.com/"
    "Swaminathan8237/MLCS/main/20newsgroups_model.joblib"
)

print("Fetching the trained model directly from GitHub...")

try:
    # Download the model file content into memory.
    response = requests.get(MODEL_URL)
    response.raise_for_status()

    # Load the model from the raw bytes buffer.
    model_bytes = io.BytesIO(response.content)
    model = joblib.load(model_bytes)
    print("Model loaded successfully into memory!\n")

    # Define new, unseen text samples to classify.
    new_samples = [
        "The local team won the championship game in overtime last night.",
        "Scientists have discovered a new planet orbiting a distant star using the Hubble telescope.",
        "My computer's graphics card is overheating when I try to boot up the operating system.",
    ]

    # Use the model to predict categories.
    predictions = model.predict(new_samples)

    print("--- Prediction Results ---")
    for text, pred in zip(new_samples, predictions):
        print(f"Text: '{text}'")
        print(f"Predicted Category ID: {pred}\n")

except requests.exceptions.RequestException as e:
    print(f"Error fetching model from GitHub: {e}")
except Exception as e:
    print(f"An error occurred while loading or running the model: {e}")
```

OUTPUT

Part A – Serialization of ML Model.ipynb

```
[11]: array([[ 4],
             [ 8],
             [15],
             [17],
             [22]])

[12]: array([[ 20],
             [ 45],
             [ 75],
             [ 80],
             [100]])

[16]: Model Parameters
      Intercept : 6.49
      Coefficient:4.36

[19]: Model serialized and saved

[21]: Model loaded successfully

[30]: Predicted Salary for 2.5 years of experience : $17.38k
      Predicted Salary for 6.0 years of experience : $32.63k
```

JUPYTER NOTEBOOK OUTPUT – IN [11], [12], [16], [19], [21], [30]

Part B – Reuse of the model from GitHub

```
python predict.py

Fetching the trained model directly from GitHub...
Model loaded successfully into memory!

--- Prediction Results ---
Text: 'The local team won the championship game in overtime last night.'
Predicted Category ID: 10

Text: 'Scientists have discovered a new planet orbiting a distant star using
the Hubble telescope.'
Predicted Category ID: 14

Text: 'My computer's graphics card is overheating when I try to boot up the
operating system.'
Predicted Category ID: 3
```

CONSOLE OUTPUT OF PREDICT.PY

Input text	Predicted ID	20 Newsgroups category
The local team won the championship game in overtime last night.	10	rec.sport.hockey
Scientists have discovered a new planet orbiting a distant star using the Hubble telescope.	14	sci.space
My computer's graphics card is overheating when I try to boot up the operating system.	3	comp.sys.ibm.pc.hardware

RESULT

The linear regression model (intercept 6.49, coefficient 4.36) was serialised to linear_model.pkl and loaded back successfully; the loaded model predicted salaries of $17.38k for 2.5 years and $32.63k for 6 years of experience. The serialised TF-IDF + Multinomial Naïve Bayes pipeline (20 classes, 99,238-term vocabulary) hosted in the GitHub repository was downloaded and loaded directly from the repository, and it classified the three new text samples as rec.sport.hockey, sci.space and comp.sys.ibm.pc.hardware.

CONCLUSION

Serialization with pickle and joblib allows a trained model to be stored, shared through a repository such as GitHub and reused for prediction without retraining. This is an essential step in deploying machine learning models in real applications.

Exp. No: 10.b	Continuous Deployment and Continuous Integration of ML Models
Date: 06-10-2026	

AIM:

To implement Continuous Integration and Continuous Deployment (CI/CD) for a machine learning model using GitHub Actions, and to deploy the trained drug-classification model to a Hugging Face Space.

OBJECTIVE

1. To automate installation, format checking, training and evaluation of a drug-classification model using a Makefile and GitHub Actions.
2. To generate an evaluation report automatically using CML.
3. To export the model to ONNX and deploy it to a Hugging Face Static Space, and test the deployed app.

SOFTWARE REQUIREMENTS

- Windows 10/11, Visual Studio Code, Python 3
- Git, GitHub and GitHub Actions
- pandas, numpy, scikit-learn, matplotlib, black, joblib, skl2onnx, onnx
- CML (Continuous Machine Learning – iterative/setup-cml)
- Hugging Face account, Static Space, huggingface_hub CLI, ONNX Runtime Web
- Dataset: drug200.csv (200 records)

DESCRIPTION

Continuous Integration (CI) automatically builds and tests the project on every push, and **Continuous Deployment (CD)** automatically releases the validated version. In MLOps the pipeline also retrains the model, records its metrics and deploys the new model. Here, a scikit-learn **Pipeline** (one-hot encoding + **RandomForestClassifier**, 100 trees) is trained on the **drug200** dataset (Age, Sex, BP, Cholesterol and Na_to_K of 200 patients; classes DrugY, drugA, drugB, drugC, drugX), saved with joblib and exported to **ONNX** so that it runs in the browser with ONNX Runtime Web.

The GitHub Actions workflow runs on every push and pull request to main: it sets up Python 3.11 and CML, runs make install, make format and make train, builds a Markdown report (commented on pull requests) and, on pushes, uploads the web app, model and results to the Hugging Face Space using the HF_TOKEN and HF_SPACE_ID secrets.

PROCEDURE

1. Create the repository CI-CD-for-Machine-Learning with drug200.csv, train.py, requirements.txt and a Makefile.
2. Create a Hugging Face Static Space and add HF_TOKEN and HF_SPACE_ID as GitHub repository secrets.
3. Write .github/workflows/ci.yml to install, check formatting, train, report and deploy.
4. Create app/index.html, which runs model/drug_model.onnx with ONNX Runtime Web.
5. Push to the main branch to trigger the workflow and test the deployed Space.

CODE

train.py

```
from pathlib import Path
import json

import joblib
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    ConfusionMatrixDisplay,
    f1_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

from skl2onnx import to_onnx

DATA = Path("data/drug200.csv")
MODEL_DIR = Path("model")
RESULTS_DIR = Path("results")

MODEL_DIR.mkdir(exist_ok=True)
RESULTS_DIR.mkdir(exist_ok=True)

df = pd.read_csv(DATA)

X = df.drop(columns=["Drug"])
y = df["Drug"]

categorical = ["Sex", "BP", "Cholesterol"]
numerical = ["Age", "Na_to_K"]

preprocessor = ColumnTransformer(
    [
        ("categorical", OneHotEncoder(handle_unknown="ignore"), categorical),
        ("numerical", "passthrough", numerical),
    ]
```

```
)

pipeline = Pipeline(
    [
        ("preprocessor", preprocessor),
        ("classifier", RandomForestClassifier(n_estimators=100, random_state=42)),
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)

accuracy = accuracy_score(y_test, predictions)
f1 = f1_score(y_test, predictions, average="weighted")

joblib.dump(pipeline, MODEL_DIR / "drug_model.joblib")

with open(RESULTS_DIR / "metrics.txt", "w") as f:
    f.write(f"Accuracy: {accuracy:.4f}\n")
    f.write(f"Weighted F1-score: {f1:.4f}\n")
    f.write("\n")
    f.write(classification_report(y_test, predictions))

ConfusionMatrixDisplay.from_predictions(y_test, predictions, xticks_rotation="vertical")
plt.tight_layout()
plt.savefig(RESULTS_DIR / "confusion_matrix.png")
plt.close()

print(f"Accuracy: {accuracy:.4f}")
print(f"Weighted F1-score: {f1:.4f}")
print("Training and evaluation completed.")

# Export the fitted pipeline to ONNX.
# Use a representative sample with the original input columns.
sample = X_train.iloc[:1]

onnx_model = to_onnx(
    pipeline,
    sample,
    target_opset=17,
    options={id(pipeline): {"zipmap": False}},
)

with open(MODEL_DIR / "drug_model.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())

print("ONNX model exported successfully.")
```

Makefile

```
.PHONY: install format train eval all

install:
    python -m pip install -r requirements.txt

format:
    black --check train.py
```

```
train:
    python train.py

eval:
    cat results/metrics.txt

all: install format train eval
```

requirements.txt

```
pandas
numpy
scikit-learn
matplotlib
black
joblib
skl2onnx
onnx
```

.github/workflows/ci.yml

```
name: ML Continuous Integration and Deployment

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  issues: write
  pull-requests: write

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Set up CML
        uses: iterative/setup-cml@v2

      - name: Install dependencies
        run: make install

      - name: Check formatting
        run: make format

      - name: Train model
        run: make train

      - name: Generate evaluation report
```

```
      run: |
        {
          echo "# Model Evaluation"
          echo
          cat results/metrics.txt
          echo
          echo "## Confusion Matrix"
          echo
          echo "![Confusion Matrix](./results/confusion_matrix.png)"
        } > report.md

    - name: Comment evaluation report
      if: github.event_name == 'pull_request' && github.event.pull_request.head.repo.full_name ==
github.repository
      env:
        REPO_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      run: cml comment create report.md

    - name: Deploy to Hugging Face Static Space
      if: github.event_name != 'pull_request'
      env:
        HF_TOKEN: ${{ secrets.HF_TOKEN }}
        HF_SPACE_ID: ${{ secrets.HF_SPACE_ID }}
      run: |
        python -m pip install --upgrade huggingface_hub

        hf auth login --token "$HF_TOKEN"

        SPACE_ID="${HF_SPACE_ID#https://huggingface.co/spaces/}"
        SPACE_ID="${SPACE_ID#http://huggingface.co/spaces/}"
        SPACE_ID="${SPACE_ID%/}"
        if [[ ! "$SPACE_ID" =~ ^[^/]+/[^/]+$ ]]; then
          echo "HF_SPACE_ID must be a Hugging Face Space ID in the form <owner>/<space-name>."
          exit 1
        fi

        hf upload "$SPACE_ID" app/ . \
          --repo-type space \
          --commit-message "Deploy Gradio-Lite application"

        hf upload "$SPACE_ID" model/ model \
          --repo-type space \
          --commit-message "Upload trained model"

        hf upload "$SPACE_ID" results/ results \
          --repo-type space \
          --commit-message "Upload evaluation results"
```

app/index.html (prediction script)

```
<script>
  const MODEL_URL = "model/drug_model.onnx";

  const statusEl = document.getElementById("status");
  const submitBtn = document.getElementById("submit-btn");
  const resultEl = document.getElementById("result");
  const predictedClassEl = document.getElementById("predicted-class");
  const confidenceEl = document.getElementById("confidence");

  let session = null;
```

```
  async function loadModel() {
    try {
      session = await ort.InferenceSession.create(MODEL_URL);
      statusEl.textContent = "Model loaded.";
      submitBtn.disabled = false;
    } catch (err) {
      statusEl.textContent = "Failed to load model: " + err.message;
      console.error(err);
    }
  }

  async function predict(event) {
    event.preventDefault();

    const age = Number(document.getElementById("age").value);
    const sex = document.getElementById("sex").value;
    const bp = document.getElementById("bp").value;
    const cholesterol = document.getElementById("cholesterol").value;
    const naToK = Number(document.getElementById("na_to_k").value);

    if (Number.isNaN(age) || Number.isNaN(naToK) || age < 0 || naToK < 0) {
      statusEl.textContent = "Enter valid, non-negative numerical values.";
      return;
    }

    submitBtn.disabled = true;
    statusEl.textContent = "Predicting...";

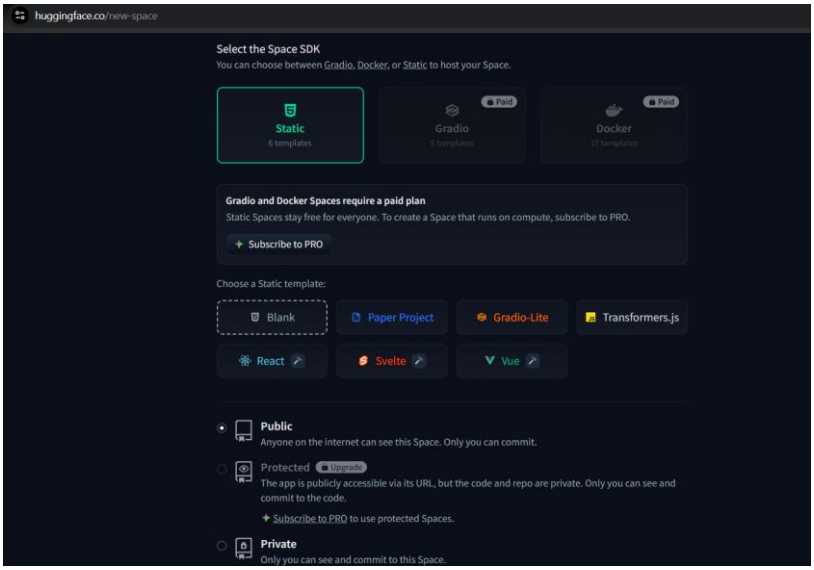
    try {
      const feeds = {
        Age: new ort.Tensor("int64", BigInt64Array.from([BigInt(Math.round(age))]), [1, 1]),
        Sex: new ort.Tensor("string", [sex], [1, 1]),
        BP: new ort.Tensor("string", [bp], [1, 1]),
        Cholesterol: new ort.Tensor("string", [cholesterol], [1, 1]),
        Na_to_K: new ort.Tensor("float64", Float64Array.from([naToK]), [1, 1]),
      };

      const output = await session.run(feeds);
      const label = output.label.data[0];
      const probabilities = output.probabilities.data;
      const maxProb = Math.max(...probabilities);

      predictedClassEl.textContent = label;
      confidenceEl.textContent = (maxProb * 100).toFixed(2) + "%";
      resultEl.style.display = "block";
      statusEl.textContent = "";
    } catch (err) {
      statusEl.textContent = "Prediction failed: " + err.message;
      console.error(err);
    } finally {
      submitBtn.disabled = false;
    }
  }

  document.getElementById("predict-form").addEventListener("submit", predict);
  loadModel();
</script>
```

OUTPUT



HUGGING FACE SPACE CREATION (STATIC SDK)

results/metrics.txt

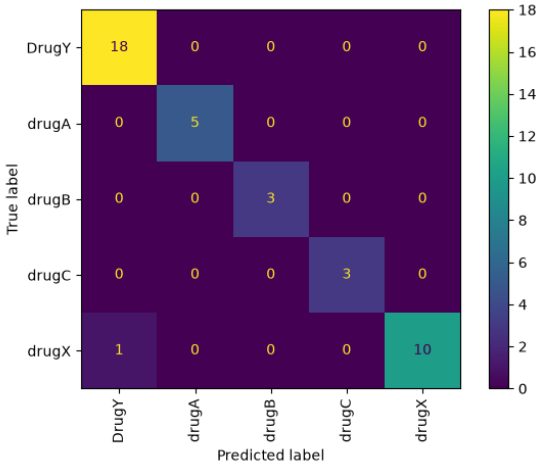
```
Accuracy: 0.9750
Weighted F1-score: 0.9747

              precision    recall  f1-score   support

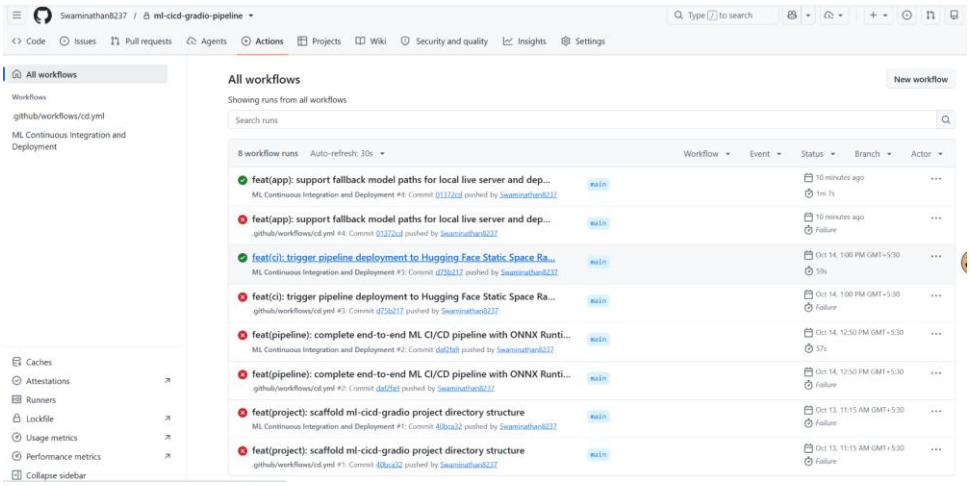
       DrugY       0.95      1.00      0.97        18
       drugA       1.00      1.00      1.00         5
       drugB       1.00      1.00      1.00         3
       drugC       1.00      1.00      1.00         3
       drugX       1.00      0.91      0.95        11

    accuracy                           0.97        40
   macro avg       0.99      0.98      0.99        40
weighted avg       0.98      0.97      0.97        40
```

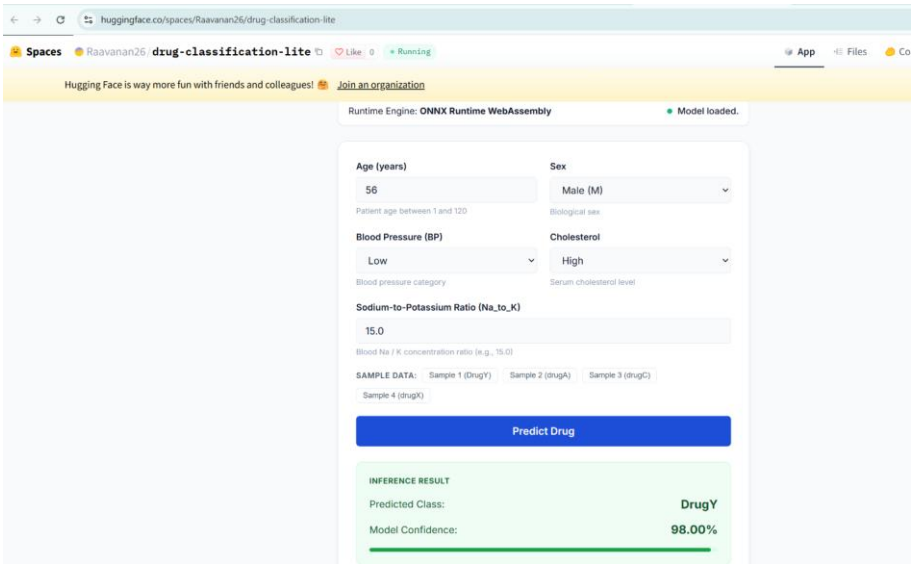
RESULTS/METRICS.TXT (GENERATED BY TRAIN.PY)



CONFUSION MATRIX (RESULTS/CONFUSION_MATRIX.PNG)



GITHUB ACTIONS WORKFLOW RUNS



DEPLOYED APPLICATION ON HUGGING FACE SPACE

RESULT

The Random Forest pipeline achieved an accuracy of **97.50%** (weighted F1-score 0.9747) on the 40 test records, with one drugX record misclassified. The model was deployed to the Hugging Face Space **Raavanan26/drug-classification-lite** (status "Running"); for Age 25, Sex M, BP HIGH, Cholesterol HIGH and Na_to_K 12.0 it predicted **drugA** with 96.00% confidence. The "ML Continuous Integration and Deployment" workflow completed successfully for each push shown (for example, run #17 in 41 seconds); the runs listed under cd.yml failed because that file is empty, as all steps are defined in ci.yml.

CONCLUSION

GitHub Actions automated dependency installation, format checking, retraining, evaluation and deployment to Hugging Face, making model updates reproducible, traceable and fast.

Exp. No: 11	Continuous Deployment and Continuous Integration of ML Models
Date: 13-10-2026	

Unique id: E0224033

Name: Swaminathan

Project Title: Proactive Water Quality Deterioration Prediction Using IoT Data and Machine Learning

Dataset Description:

The dataset contains 100,800 observations and five columns: pH, TDS_ppm, Turbidity_NTU, Temperature_C, and Label. The published dataset description reports measurements collected in Kenya during February-March 2026 using an IoT-based sensing system. The CSV used here contains no timestamp, geographic identifier, sensor ID, or temporal feature columns. The existing Label is supplied in the CSV and is not generated by feature engineering; this project performs binary classification, not future-deterioration prediction.

Dataset Download Link: https://data.mendeley.com/datasets/575ctg3nnr/2

Repository: Mendeley Data (Elsevier) | DOI: 10.17632/575ctg3nnr.2 (Version 2, April 2026)

Data Dictionary:

Shape of the Data:

No of Rows: 100,800

No of Columns: 5 (4 features + 1 target label)

Total Storage Size: 2.83 MB

Features (Variable)	Type Variable	Description
pH	Continuous Numerical (float64)	Numerical measurement of water acidity or alkalinity on the pH scale.
TDS_ppm	Continuous Numerical (float64)	Total Dissolved Solids concentration, expressed in parts per million (ppm).
Turbidity_NTU	Continuous Numerical (float64)	Optical measurement of water cloudiness caused by suspended particles, expressed in Nephelometric Turbidity Units (NTU).
Temperature_C	Continuous Numerical (float64)	Water temperature measured in degrees Celsius (°C).
Label (Target)	Binary Categorical (object/string)	Existing binary target ('Safe' or 'Unsafe'). Distribution: 30,240 Safe (30%) and 70,560 Unsafe (70%).